DbCopy：資料庫複製工具

向皓田

2026-03-17

Table of contents

DbCopy：資料庫複製工具 **8**
— 從零開始的系統設計與實作完整技術指南 — 8
目錄 8

1 為什麼需要 DbCopy？ **9**
1.1 資料庫遷移的痛點 9
1.1.1 常見的困境 9
1.2 現有工具的不足 9
1.3 DbCopy 的設計目標 10
1.3.1 目標一：完整的物件覆蓋 10
1.3.2 目標二：智慧的相依性分析 10
1.3.3 目標三：分階段的複製控制 10
1.3.4 目標四：零安裝部署 10
1.3.5 目標五：跨平台支援 11
1.4 本書適合的讀者 11

2 技術選型與架構總覽 **12**
2.1 為什麼選擇 ASP.NET Core 12
2.2 為什麼用 Dapper 而不是 Entity Framework 12
2.2.1 Entity Framework 不適合的原因 12
2.2.2 Dapper 的優勢 13
2.3 前端技術選擇：Bootstrap + jQuery 13
2.4 整體架構圖與分層設計 13
2.4.1 各層職責 14
2.4.2 資料流 15
2.5 專案目錄結構 15
2.5.1 設計原則 16

3 資料庫物件的抽象建模 **17**
3.1 資料庫物件有哪些種類 17
3.2 設計 DbObjectType 列舉 17
3.2.1 列舉值的順序至關重要 18
3.3 設計 DbObject 模型 18
3.3.1 設計決策解析 18
3.4 設計 DbIndex 與 SyncStatus 19
3.4.1 DbIndex — 索引模型 19
3.4.2 SyncStatus — 同步狀態模型 20
3.5 設計 API 請求模型 21
3.5.1 DbConnectionInfo — 連線資訊 21

3.5.2 CompareRequest 與 CopyRequest . . . 21
3.6 列舉排序的重要性：依賴順序 . . . 22

4 介面導向設計 — IDbService 23
4.1 為什麼需要介面 . . . 23
4.1.1 為什麼不用抽象類別？ . . . 23
4.2 IDbService 的九個方法 . . . 24
4.3 每個方法的職責與契約 . . . 24
4.3.1 TestConnectionAsync — 連線測試 . . . 24
4.3.2 GetDbObjectsAsync — 物件清單 . . . 25
4.3.3 GetObjectDefinitionAsync — DDL 定義 . . . 25
4.3.4 CheckObjectExistsAsync — 存在性檢查 . . . 25
4.3.5 EnsureSchemaExistsAsync — Schema 建立 . . . 25
4.3.6 GetDependenciesAsync — 相依性分析 . . . 25
4.3.7 GetRowCountAsync — 資料表行數 . . . 26
4.3.8 GetTableIndexesAsync — 索引清單 . . . 26
4.3.9 CopyObjectAsync — 複製操作 . . . 26
4.4 如何在未來擴展新的資料庫類型 . . . 27
4.4.1 目前架構的一個務實折衷 . . . 28

5 Dapper 擴充方法與結構化日誌 29
5.1 為什麼要包裝 Dapper . . . 29
5.2 DapperExtensions 的設計 . . . 29
5.2.1 設計要點 . . . 30
5.3 LoggerMessage 原始碼產生器 . . . 31
5.3.1 為什麼不直接用 `logger.LogInformation()`？ . . . 31
5.3.2 partial class 與 partial method . . . 31
5.4 確保每條 SQL 恰好記錄一次 . . . 32
5.4.1 繞過 Dapper 的情況 . . . 32
5.4.2 日誌輸出範例 . . . 33

6 SQL Server 服務實作 34
6.1 連線測試 . . . 34
6.2 物件清單查詢 — sys.* 目錄檢視 . . . 34
6.2.1 SQL Server 系統目錄速覽 . . . 35
6.2.2 函數的三種子類型 . . . 35
6.2.3 Type 欄位的字串映射 . . . 36
6.3 DDL 定義重建 . . . 36
6.3.1 為什麼有些物件可以用 OBJECT_DEFINITION()，有些不行？ . . . 36
6.3.2 資料表定義重建 . . . 36
6.3.3 序列定義重建 . . . 37
6.4 型別長度的位元組與字元換算 . . . 38
6.4.1 注意：兩個不同的長度來源 . . . 39
6.5 相依性分析 . . . 39
6.5.1 依賴分析的三個維度 . . . 40

6.6 SqlBulkCopy 大量資料複製 . . . 40
6.6.1 SqlBulkCopy 的關鍵選項 . . . 41
6.6.2 串流模式 . . . 41
6.7 IDENTITY 欄位重設 . . . 41
6.7.1 為什麼需要重設 IDENTITY？ . . . 42
6.8 索引與外鍵的建立 . . . 42
6.8.1 索引定義提取 . . . 42
6.8.2 外鍵定義提取 . . . 43
6.8.3 容錯設計 . . . 43

7 PostgreSQL 服務實作 44
7.1 pg_catalog 與 information_schema 的選擇 . . . 44
7.2 排除 Extension 擁有的物件 . . . 44
7.3 使用者定義型別的四種子類型 . . . 45
7.3.1 Enum 型別 . . . 45
7.3.2 Composite 型別（複合型別） . . . 46
7.3.3 Domain 型別 . . . 46
7.3.4 Base 型別（Shell 定義） . . . 46
7.3.5 自動偵測型別子類型 . . . 47
7.4 COPY 協定：二進位模式與文字模式 . . . 47
7.4.1 二進位模式（Binary Format） . . . 47
7.4.2 文字模式（Text Format） . . . 48
7.4.3 模式選擇邏輯 . . . 48
7.5 空間資料的特殊處理 . . . 49
7.6 Extension 自動偵測與安裝 . . . 49
7.6.1 偵測規則 . . . 50
7.7 序列值重設 . . . 50
7.7.1 設計要點 . . . 51

8 RESTful API 設計 — DbController 52
8.1 API 端點規劃 . . . 52
8.1.1 為什麼全部用 POST？ . . . 52
8.1.2 路由配置 . . . 52
8.2 Primary Constructor 注入 . . . 53
8.2.1 Primary Constructor 的優勢 . . . 53
8.2.2 GetService — 服務分派 . . . 54
8.3 四大端點的實作 . . . 54
8.3.1 test — 連線測試 . . . 54
8.3.2 objects — 列出物件 . . . 54
8.3.3 compare — 比較差異（核心端點） . . . 55
8.3.4 copy — 複製物件 . . . 56
8.4 平行化查詢策略 . . . 56
8.4.1 第一層平行化：存在性檢查 + 相依性分析 . . . 57
8.4.2 第二層平行化：資料表的四個查詢 . . . 57
8.4.3 Safe Wrapper 模式 . . . 57

8.5 錯誤處理與回應格式 . . . 58
8.5.1 回應格式 . . . 58

9 單頁式 Web 介面 59
9.1 Razor Pages 與前端 JavaScript 的協作 . . . 59
9.1.1 為什麼不用 Razor 的 OnPost Handler？ . . . 59
9.2 連線管理系統 . . . 59
9.2.1 資料結構 . . . 59
9.2.2 唯讀連線 . . . 60
9.2.3 匯入匯出 . . . 60
9.3 分析與比較流程 . . . 61
9.4 樹狀結構的結果表格 . . . 62
9.4.1 分組邏輯 . . . 62
9.4.2 展開/收合 . . . 63
9.4.3 勾選機制 . . . 63
9.5 批次同步與進度追蹤 . . . 64
9.5.1 同步流程 . . . 64
9.5.2 中斷機制 . . . 65
9.5.3 進度條 . . . 65
9.6 匯入匯出連線設定 . . . 66
9.6.1 自訂確認對話框 . . . 66
9.6.2 篩選系統 . . . 67
9.6.3 統計列 . . . 67

10 CSS 主題設計 68
10.1 CSS 變數系統 . . . 68
10.1.1 色彩系統的設計原則 . . . 69
10.2 漸層配色方案 . . . 69
10.2.1 背景漸層 . . . 69
10.2.2 按鈕漸層 . . . 69
10.2.3 表頭漸層 . . . 70
10.2.4 捲軸漸層 . . . 70
10.3 物件類型色彩編碼 . . . 70
10.3.1 樹狀表格的列樣式 . . . 71
10.3.2 狀態 Badge . . . 71
10.4 響應式設計考量 . . . 72
10.4.1 字型大小適應 . . . 72
10.4.2 表格滾動區域 . . . 72
10.4.3 字型選擇 . . . 73
10.4.4 Modal 對話框 . . . 73

11 Program.cs — 應用程式進入點 74
11.1 Serilog 日誌配置 . . . 74
11.1.1 為什麼選擇 Serilog？ . . . 74
11.1.2 日誌輸出配置 . . . 74

11.2 依賴注入容器設定 . . . 75
11.2.1 生命週期選擇：Scoped . . . 75
11.2.2 中介軟體管線 . . . 75
11.3 嵌入式靜態檔案 . . . 76
11.3.1 問題 . . . 76
11.3.2 解決方案 . . . 76
11.3.3 運作原理 . . . 77
11.3.4 需要的 NuGet 套件 . . . 77
11.4 連接埠衝突自動偵測 . . . 77
11.4.1 核心演算法 . . . 77
11.4.2 連接埠可用性檢查 . . . 78
11.4.3 reservedPorts 的作用 . . . 79
11.5 自動開啟瀏覽器 . . . 79
11.5.1 設計要點 . . . 79
11.5.2 跨平台相容性 . . . 79
11.5.3 全域錯誤處理 . . . 80

12 跨平台建置與發佈 81
12.1 專案檔設定解析 . . . 81
12.1.1 各設定項解析 . . . 81
12.1.2 版本號的萬用字元 . . . 82
12.1.3 套件依賴 . . . 82
12.2 五平台單一執行檔發佈 . . . 82
12.2.1 發佈參數解析 . . . 83
12.2.2 五個目標平台 . . . 83
12.2.3 產出結構 . . . 84
12.3 語意化版本控制 . . . 84
12.3.1 安全機制 . . . 85
12.4 嵌入式資源的運作原理 . . . 85
12.4.1 建置時流程 . . . 85
12.4.2 執行時流程 . . . 86
12.4.3 與 Single File Publishing 的配合 . . . 86

13 安全性設計 87
13.1 SQL 注入防護 . . . 87
13.1.1 第一層：參數化查詢（Dapper） . . . 87
13.1.2 哪些場景無法使用參數化？ . . . 87
13.2 識別符跳脫策略 . . . 88
13.2.1 SQL Server：方括號跳脫 . . . 88
13.2.2 PostgreSQL：雙引號跳脫 . . . 88
13.2.3 PostgreSQL 特有：PL/pgSQL 中的 format(‘%I’, …) . . . 89
13.2.4 QUOTENAME — SQL Server 內建的安全識別符 . . . 89
13.3 連線字串的安全考量 . . . 89
13.3.1 傳輸安全 . . . 89
13.3.2 未實作認證機制 . . . 90
13.3.3 唯讀連線的保護 . . . 90

13.3.4 DDL 語句的安全性 . . . 90
13.3.5 安全設計的哲學 . . . 91

14 效能與可靠性 92
14.1 非同步設計模式 . . . 92
14.1.1 為什麼非同步很重要？ . . . 92
14.1.2 `await using` 模式 . . . 93
14.2 Task.WhenAll 平行化策略 . . . 93
14.2.1 策略一：來源與目標平行查詢 . . . 93
14.2.2 策略二：四個資料表查詢平行化 . . . 93
14.2.3 條件性跳過 . . . 94
14.2.4 注意事項 . . . 94
14.3 批次處理與串流 . . . 94
14.3.1 SQL Server：SqlBulkCopy 的批次模式 . . . 94
14.3.2 PostgreSQL：COPY 協定的串流模式 . . . 94
14.3.3 記憶體使用比較 . . . 95
14.4 優雅的部分失敗處理 . . . 95
14.4.1 索引建立的容錯 . . . 95
14.4.2 外鍵建立的特殊處理 . . . 96
14.4.3 行數查詢的容錯 . . . 96
14.4.4 IDENTITY/序列重設的容錯 . . . 96
14.4.5 容錯層級總結 . . . 97

15 四階段複製流程的設計哲學 98
15.1 為什麼分階段 . . . 98
15.1.1 一步到位的問題 . . . 98
15.1.2 分階段的優勢 . . . 99
15.2 Phase 1：結構 . . . 99
15.2.1 為什麼主鍵在 Phase 1 而不是 Phase 3？ . . . 99
15.2.2 Phase 1 的前置條件 . . . 99
15.3 Phase 2：資料 . . . 100
15.3.1 為什麼 Phase 2 在 Phase 3 之前？ . . . 100
15.3.2 Phase 2 的後處理 . . . 100
15.4 Phase 3：索引 . . . 100
15.4.1 容錯設計 . . . 101
15.5 Phase 4：外鍵 . . . 101
15.5.1 為什麼外鍵是最後一個階段？ . . . 101
15.5.2 特殊錯誤處理 . . . 101
15.6 相依性排序與複製順序 . . . 102
15.6.1 列舉值蘊含的順序 . . . 102
15.6.2 建議的複製順序 . . . 102
15.6.3 相依性資訊的呈現 . . . 103
15.6.4 完整流程圖 . . . 103
15.6.5 設計的自洽性 . . . 104
15.7 本書總結 . . . 104

DbCopy：資料庫複製工具

— 從零開始的系統設計與實作完整技術指南 —

目錄

章節	標題	說明
第一章	為什麼需要 DbCopy	需求背景與問題分析
第二章	技術選型與架構總覽	技術選型、分層架構、系統總覽
第三章	資料庫物件的抽象建模	資料庫結構的模型設計
第四章	介面導向設計	以介面抽象實現多資料庫支援
第五章	Dapper 擴充方法與結構化日誌	Dapper 擴充與日誌實作
第六章	SQL Server 服務實作	SQL Server 資料庫服務的完整實作
第七章	PostgreSQL 服務實作	PostgreSQL 資料庫服務的完整實作
第八章	RESTful API 設計	Web API 端點設計與實作
第九章	單頁式 Web 介面	前端 SPA 介面設計
第十章	CSS 主題設計	視覺風格與主題樣式
第十一章	應用程式進入點	Program.cs 與啟動流程
第十二章	跨平台建置與發佈	多平台編譯、打包與發佈
第十三章	安全性設計	安全防護與最佳實踐
第十四章	效能與可靠性	效能優化與穩定性設計
第十五章	四階段複製流程	完整的資料庫複製執行流程

作者備註

本書以 DbCopy 系統的實際原始碼為基礎，從設計動機到實作細節完整呈現。所有程式碼範例均取自真實的 production codebase，而非簡化的教學版本。

C# 程式碼採用全中文命名，資料庫維持英文命名——這是本專案的設計準則。

1 為什麼需要 DbCopy？

1.1 資料庫遷移的痛點

在軟體開發的日常工作中，資料庫遷移與同步是最常見卻也最令人頭痛的任務之一。無論是：

- **開發環境同步**：將正式環境的資料庫結構複製到本地開發環境
- **測試環境準備**：為 QA 團隊建立一份與生產環境結構一致的測試資料庫
- **跨伺服器遷移**：將舊伺服器的資料庫搬移到新伺服器
- **Schema 版本對齊**：確保多個環境的資料庫結構保持一致

這些工作往往需要 DBA 或資深開發者手動操作，過程繁瑣且容易出錯。

1.1.1 常見的困境

1. **物件繁多**：一個中型資料庫可能包含數百張資料表、數十個檢視、上百個預存程序，手動逐一比對幾乎不可能。
2. **相依性複雜**：資料表之間有外鍵關聯，預存程序參考檢視，函數使用自訂型別——物件之間的相依性形成了一張複雜的有向圖，複製順序稍有不對就會失敗。
3. **資料量龐大**：動輒數百萬筆的資料表，傳統的 INSERT 語句效率低落，需要專門的大量載入工具。
4. **人為錯誤**：手動執行 DDL 腳本時，漏掉一個索引、忘記建立一個自訂型別，往往要到應用程式報錯時才會發現。

1.2 現有工具的不足

市面上並非沒有資料庫比較與同步工具，例如：

工具	限制
SQL Server Management Studio (SSMS)	僅支援 SQL Server，且結構比較功能有限
pgAdmin	僅支援 PostgreSQL，無自動同步功能
Redgate SQL Compare	商業授權費用高昂，僅支援 SQL Server
dbForge Schema Compare	商業軟體，單一資料庫引擎
Liquibase / Flyway	專注於版本控制遷移，非即時比較同步

這些工具各有其價值，但在以下場景往往力有未逮：

- 需要**跨資料庫引擎**的統一介面（同時處理 SQL Server 和 PostgreSQL）
- 需要**輕量化**的部署（一個執行檔即可運作，無需安裝）
- 需要**分階段控制**的同步（先結構、再資料、再索引、最後外鍵）
- 需要 **Web 介面**方便遠端操作（不需要安裝桌面軟體）

1.3 DbCopy 的設計目標

DbCopy 的誕生源自於一個簡單的需求：**我需要一個工具，能夠快速比較兩個資料庫的差異，並將缺少的物件複製過去。**

基於這個核心需求，我們定義了以下設計目標：

1.3.1 目標一：完整的物件覆蓋

不只是資料表，還要支援：- 資料表（Tables）- 檢視（Views）- 預存程序（Stored Procedures）- 函數（Functions）- 使用者定義型別（User-Defined Types）- 使用者定義資料表型別（User-Defined Table Types）- 序列（Sequences）

1.3.2 目標二：智慧的相依性分析

系統必須自動分析物件之間的相依關係，告知使用者「這個檢視依賴哪些資料表」、「這個預存程序使用了哪些自訂型別」。

1.3.3 目標三：分階段的複製控制

資料表的複製分為四個階段：1. **結構**（Schema）：建立資料表定義 2. **資料**（Data）：大量複製資料列 3. **索引**（Indexes）：建立非主鍵索引 4. **外鍵**（Foreign Keys）：建立外部參照

使用者可以選擇一次執行全部階段，或逐階段手動控制。

1.3.4 目標四：零安裝部署

透過.NET 的 Single File Publishing 技術，將整個應用程式（包含 Web 伺服器、靜態檔案、所有依賴套件）打包成一個單一執行檔。使用者只需要下載、雙擊執行，瀏覽器就會自動開啟。

1.3.5 目標五：跨平台支援

同一套程式碼可以建置為五個平台的執行檔：- Windows x64 - Linux x64 - Linux ARM64 - macOS x64（Intel）- macOS ARM64（Apple Silicon）

1.4 本書適合的讀者

本書適合以下背景的讀者：

- **中高階.NET 開發者**：已有 C# 基礎，想學習如何設計完整的工具型應用程式
- **資料庫管理員（DBA）**：想了解資料庫物件的程式化管理方式
- **系統架構師**：想參考介面導向設計、分層架構、跨資料庫抽象的實務做法
- **全端開發者**：想學習 ASP.NET Core + Razor Pages + JavaScript 的整合方式

本書假設讀者已具備以下基礎知識：- C# 語法基礎 - SQL 基礎（SELECT、CREATE TABLE 等）- HTTP 與 REST API 的基本概念 - HTML/CSS/JavaScript 的基本知識

不需要事先了解的內容（本書會從頭講解）：- ASP.NET Core 的中介軟體管線 - Dapper ORM 的使用方式 - SQL Server 與 PostgreSQL 的系統目錄 - 大量資料載入（Bulk Copy）技術 - Single File Publishing 的原理

下一章預告：在開始寫程式之前，我們需要先做好技術選型。第 2 章將討論為什麼選擇 ASP.NET Core 作為框架、為什麼用 Dapper 而不是 Entity Framework，以及整體架構的分層設計。

2 技術選型與架構總覽

2.1 為什麼選擇 ASP.NET Core

DbCopy 是一個需要使用者介面的工具。在技術選型時，我們面臨了兩個方向：

方案	優點	缺點
桌面應用（WPF/WinForms）	原生體驗、豐富控制元件	僅 Windows、部署複雜
Web 應用（ASP.NET Core）	跨平台、零安裝、瀏覽器即用	需要內建 Web 伺服器
命令列工具（Console App）	最輕量	互動體驗差、難以呈現表格比較

最終選擇 **ASP.NET Core** 的原因：

1. **跨平台**：.NET 10 原生支援 Windows、Linux、macOS，一套程式碼打天下。
2. **Self-Hosted**：Kestrel 內建 Web 伺服器，不需要 IIS 或 Nginx，單一執行檔即可運行。
3. **內建 DI 容器**：依賴注入是框架內建功能，不需要額外引入 IoC 容器。
4. **嵌入式靜態檔案**：透過 `ManifestEmbeddedFileProvider`，所有前端資源可以嵌入到執行檔中。
5. **Razor Pages + Controllers 混合架構**：頁面用 Razor Pages 渲染、API 用 Controllers 處理，各司其職。

2.2 為什麼用 Dapper 而不是 Entity Framework

DbCopy 的核心工作是「讀取資料庫的系統目錄（System Catalog）」，這與一般的 CRUD 應用完全不同。

2.2.1 Entity Framework 不適合的原因

`sys.tables`、`sys.columns`、`pg_class`、`pg_attribute...`

這些系統目錄：- **沒有固定的 Entity Model**：我們查詢的是中繼資料，不是業務資料 - **需要大量原始 SQL**：因為查詢系統目錄需要精確控制 SQL 語法 - **回傳結構多變**：不同的查詢回傳完全不同的欄位組合 - **效能優先**：不需要 Change Tracking、Lazy Loading 等 ORM 功能

2.2.2 Dapper 的優勢

Dapper 是一個「微型 ORM」（Micro-ORM），它的設計哲學是：

> 你寫 SQL，我幫你把結果映射到 C# 物件。

```
// Dapper 的典型用法 —簡潔、直覺、零魔法
var results = await conn.QueryAsync<DbObject>(sql, new { Schema, Name });
```

- **零學習曲線**：會寫 SQL 就能用
- **零效能損耗**：幾乎等同於直接使用 ADO.NET
- **彈性映射**：支援強型別和 `dynamic` 兩種回傳模式
- **參數化查詢**：自動處理 SQL 參數，防止注入攻擊

2.3 前端技術選擇：Bootstrap + jQuery

在前端技術的選擇上，我們刻意選擇了「經典組合」而非現代 SPA 框架：

考量	選擇	理由
CSS 框架	Bootstrap 5	元件豐富、學習成本低、單頁工具不需要設計系統
JS 框架	jQuery	DOM 操作簡單直覺、不需要建置工具鏈
圖示	Bootstrap Icons	與 Bootstrap 無縫整合
字型	Google Fonts (Inter)	專為 UI 設計的現代無襯線字體

為什麼不用 React/Vue/Angular？

DbCopy 本質上是一個**單頁工具**，不是一個複雜的 Web 應用程式。它只有：- 一個主頁面 - 幾個 Modal 對話框 - 一個動態表格

引入 SPA 框架意味著：- 需要 Node.js 建置工具鏈 - 需要額外的打包步驟 - 增加部署複雜度 - 學習成本不成比例

對於這種規模的工具，jQuery 的直接 DOM 操作反而更簡單、更快速。

2.4 整體架構圖與分層設計

DbCopy 採用經典的**三層式架構**：

```
                 前端 UI 層

Index.cshtml      site.css         jQuery
(Razor Page)      (Theme)          (Logic)


                 API 控制器層

             DbController
 POST /api/db/test
 POST /api/db/objects
 POST /api/db/compare
 POST /api/db/copy


                 服務層

           IDbService（介面）


SqlServerService    PostgreSqlService


        DapperExtensions（日誌包裝）


                 領域模型層

DbType     DbObject   SyncStatus
(Enum)     DbIndex     Request


                 基礎設施層

Serilog     Dapper     EmbeddedFileProvider
```

2.4.1 各層職責

層	職責	代表檔案
前端 UI	使用者互動、資料呈現	`Index.cshtml`, `site.css`
API 控制器	接收 HTTP 請求、協調服務	`DbController.cs`

層	職責	代表檔案
服務	封裝資料庫操作邏輯	SqlServerService.cs, PostgreSqlService.cs
領域模型	定義資料結構與列舉	DbModels.cs
基礎設施	日誌、ORM、檔案提供	DapperExtensions.cs, Program.cs

2.4.2 資料流

一次典型的「比較」操作，資料流如下：

```
使用者點擊「分析與比較」
    ↓
JavaScript 發送 POST /api/db/compare
    ↓
DbController.Compare() 接收請求
    ↓
根據 DbType 取得對應的 IDbService 實作
    ↓
Service.GetDbObjectsAsync() 查詢來源物件清單
    ↓
對每個物件，平行執行：
    Service.CheckObjectExistsAsync()  (目標)
    Service.GetDependenciesAsync()    (來源)
    ↓
若為資料表，額外平行查詢：
    Service.GetRowCountAsync()       (來源 + 目標)
    Service.GetTableIndexesAsync()   (來源 + 目標)
    ↓
組裝 SyncStatus 清單回傳
    ↓
JavaScript 渲染樹狀表格
```

2.5 專案目錄結構

```
DbCopy/
  Controllers/           # API 控制器
     DbController.cs
  Models/                # 領域模型
     DbModels.cs
  Services/              # 服務層
     IDbService.cs      # 介面定義
```

```
    DapperExtensions.cs # Dapper 日誌包裝
    SqlServerService.cs # SQL Server 實作
    PostgreSqlService.cs # PostgreSQL 實作
Pages/                  # Razor Pages
    Index.cshtml        # 主頁面
    Error.cshtml        # 錯誤頁面
    Shared/
        _Layout.cshtml # 版面配置
wwwroot/                # 靜態資源（會嵌入執行檔）
    css/site.css        # 自訂樣式
    js/site.js          # 自訂腳本
    lib/                # 第三方函式庫
Program.cs              # 應用程式進入點
DbCopy.csproj           # 專案檔
publish.sh              # 跨平台發佈腳本
release.sh              # 語意化版本腳本
```

2.5.1 設計原則

1. **單一職責**：每個檔案只負責一件事。`DbModels.cs` 只定義模型、`DbController.cs` 只處理路由。
2. **依賴反轉**：控制器依賴 `IDbService` 介面，而非具體實作。
3. **最小化檔案數**：整個專案只有 7 個 C# 檔案。不過度拆分，不建立無用的抽象層。

下一章預告：架構確定後，接下來要設計最核心的資料結構——如何用 C# 的型別系統來表達「資料庫物件」這個概念。

3 資料庫物件的抽象建模

在開始寫任何服務邏輯之前，我們需要先回答一個根本問題：**如何用 C# 的型別系統來表達「資料庫物件」這個概念？**

好的模型設計是整個系統的基石。模型設計得好，後續的服務層、API 層、前端層都會自然而然地變得簡潔。

3.1 資料庫物件有哪些種類

不論是 SQL Server 還是 PostgreSQL，資料庫中的物件大致可以分為以下幾類：

類別	SQL Server	PostgreSQL
資料表	`sys.tables`	`pg_class (relkind='r','p')`
檢視	`sys.views`	`pg_class (relkind='v','m')`
預存程序	`sys.procedures`	`pg_proc (prokind='p')`
函數	`sys.objects (FN,IF,TF)`	`pg_proc (prokind='f')`
使用者定義型別	`sys.types`	`pg_type (typtype='c','e','d')`
使用者定義資料表型別	`sys.table_types`	*(無直接對應)*
序列	`sys.sequences`	`pg_class (relkind='S')`

DbCopy 需要一個**統一的抽象**，讓同一套 API 和 UI 能夠處理所有這些物件類型。

3.2 設計 DbObjectType 列舉

```
public enum DbObjectType
{
    UserDefinedType,        // 0 —使用者定義型別
    UserDefinedTableType,   // 1 —使用者定義資料表型別
    Sequence,               // 2 —序列
    Table,                  // 3 —資料表
    View,                   // 4 —檢視
    Procedure,              // 5 —預存程序
    Function                // 6 —函數
}
```

3.2.1 列舉值的順序至關重要

注意列舉值的排列順序：**UserDefinedType □ UserDefinedTableType □ Sequence □ Table □ View □ Procedure □ Function**。

這個順序不是隨意決定的，而是反映了**物件之間的依賴關係**：

```
UserDefinedType (0)       ← 最底層，不依賴其他自訂物件
    ↑
UserDefinedTableType (1) ← 可能使用 UDT 作為欄位型別
    ↑
Sequence (2)              ← 獨立物件，但資料表的 IDENTITY/serial 可能依賴它
    ↑
Table (3)                 ← 可能使用 UDT 作為欄位型別，可能使用 Sequence
    ↑
View (4)                  ← 依賴 Table 或其他 View
    ↑
Procedure (5)             ← 可能依賴 Table、View、UDT
    ↑
Function (6)              ← 可能依賴 Table、View、UDT
```

當前端 UI 依照列舉值排序顯示物件時，自然就會呈現出正確的複製順序：先建立型別，再建立序列，然後是資料表，最後是檢視和程式碼物件。

3.3 設計 DbObject 模型

```
public class DbObject
{
    public string Schema { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public DbObjectType Type { get; set; }
    public string? Definition { get; set; }

    public string FullName => string.IsNullOrEmpty(Schema) ? Name : $"{Schema}.{Name}";
    public List<DbIndex> Indexes { get; set; } = [];
}
```

3.3.1 設計決策解析

Schema + Name 的組合

資料庫物件的唯一識別不是只靠名稱，而是 Schema 加上名稱。在 SQL Server 中，`dbo.Users` 和 `hr.Users` 是兩個不同的物件。因此我們用 `Schema` 和 `Name` 兩個屬性來識別物件。

`FullName` 是一個計算屬性，提供 `"schema.name"` 格式的便利存取。

Definition 為可空型別

`Definition` 存放物件的 DDL 定義（例如 `CREATE TABLE ...`）。它設計為 `string?`（可空），因為：

1. 在**比較階段**，我們只需要知道物件的名稱和類型，不需要取得完整定義
2. 在**複製階段**，才會按需載入定義

這是一種**延遲載入**的設計思維——不在列表查詢時就載入所有定義，避免不必要的效能開銷。

Indexes 集合

索引是資料表的附屬物件，而非獨立的資料庫物件。因此它作為 `DbObject` 的一個屬性存在，而不是作為 `DbObjectType` 的一個值。

使用 `[]`（C# 12 的集合運算式）初始化為空列表，避免 null 檢查。

3.4 設計 DbIndex 與 SyncStatus

3.4.1 DbIndex — 索引模型

```
public class DbIndex
{
    public string Name { get; set; } = string.Empty;
    public string Columns { get; set; } = string.Empty;
    public bool IsUnique { get; set; }
    public bool ExistsInDestination { get; set; }
}
```

`DbIndex` 模型的設計刻意保持簡單：

- `Name`：索引名稱，用於在來源和目標之間比對
- `Columns`：索引鍵欄位的字串表示（包含 INCLUDE 欄位）
- `IsUnique`：是否為唯一索引
- `ExistsInDestination`：比較階段由 Controller 設定，告知 UI 此索引是否需要同步

注意 `ExistsInDestination` 不是索引本身的屬性，而是**比較結果**。這是一個務實的設計選擇——把比較結果直接附著在模型上，避免建立額外的對應表。

3.4.2 SyncStatus — 同步狀態模型

```
public class SyncStatus
{
    public DbObject SourceObject { get; set; } = null!;
    public bool ExistsInDestination { get; set; }
    public string Status { get; set; } = "Pending";
    public string? Message { get; set; }
    public List<string> Dependencies { get; set; } = [];
    public long? SourceRowCount { get; set; }
    public long? TargetRowCount { get; set; }
}
```

`SyncStatus` 是**比較操作的結果模型**，它回答了以下問題：

屬性	回答的問題
`SourceObject`	來源有什麼物件？
`ExistsInDestination`	目標有沒有這個物件？
`Status`	目前的同步狀態是什麼？(Pending / Exists / Success / Error)
`Message`	如果出錯了，錯誤訊息是什麼？
`Dependencies`	這個物件依賴哪些其他物件？
`SourceRowCount` / `TargetRowCount`	來源和目標各有多少筆資料？(僅資料表)

Status 的狀態機

```
初始狀態
    ↓
Pending（目標不存在，等待複製）
    ↓ 執行複製
Success（複製成功）  或  Error（複製失敗）

Exists（目標已存在，無需複製）
```

RowCount 為 `long?`

- `long` 而非 `int`：因為資料表可能超過 21 億列（SQL Server 使用 `COUNT_BIG`）
- `?`（可空）：因為只有資料表才有行數，非資料表物件不適用此屬性；且查詢失敗時也可能返回 null

3.5 設計 API 請求模型

3.5.1 DbConnectionInfo — 連線資訊

```
public class DbConnectionInfo
{
    public string Id { get; set; } = Guid.NewGuid().ToString();
    public string Name { get; set; } = string.Empty;
    public DbType Type { get; set; }
    public string ConnectionString { get; set; } = string.Empty;
}
```

`Id` 使用 GUID 自動產生，確保即使名稱重複也能唯一識別。`DbType` 是一個簡單的列舉：

```
public enum DbType
{
    SqlServer,     // 0
    PostgreSql     // 1
}
```

3.5.2 CompareRequest 與 CopyRequest

```
public class CompareRequest
{
    public DbConnectionInfo Source { get; set; } = null!;
    public DbConnectionInfo Target { get; set; } = null!;
}

public class CopyRequest
{
    public DbConnectionInfo Source { get; set; } = null!;
    public DbConnectionInfo Target { get; set; } = null!;
    public DbObject Object { get; set; } = null!;
    public int Phase { get; set; } = 0;
    public int BatchSize { get; set; } = 1000;
}
```

設計要點：

1. **`null!` 的使用**：這些屬性在反序列化時由 JSON 填入，使用 `null!` 告訴編譯器「我知道它在建構時是 null，但使用時不會是 null」。
2. **Phase 預設為 0**：Phase 0 表示「全部階段」，是最常用的情境。
3. **BatchSize 預設為 1000**：在效能和記憶體之間取得平衡的預設值。

3.6 列舉排序的重要性：依賴順序

讓我們用一個實際的例子來說明為什麼列舉順序這麼重要。

假設來源資料庫有以下物件：

```sql
-- 1. 自訂型別
CREATE TYPE dbo.PhoneNumber FROM VARCHAR(20) NOT NULL;

-- 2. 資料表（使用了自訂型別）
CREATE TABLE dbo.Contacts (
    Id INT IDENTITY PRIMARY KEY,
    Name NVARCHAR(100),
    Phone dbo.PhoneNumber    -- 依賴 UDT
);

-- 3. 檢視（依賴資料表）
CREATE VIEW dbo.vContactList AS
SELECT Id, Name, Phone FROM dbo.Contacts;

-- 4. 預存程序（依賴資料表）
CREATE PROCEDURE dbo.GetContact @Id INT AS
SELECT * FROM dbo.Contacts WHERE Id = @Id;
```

正確的複製順序必須是：

```
1. PhoneNumber (UserDefinedType, 列舉值 0)
2. Contacts    (Table, 列舉值 3)
3. vContactList (View, 列舉值 4)
4. GetContact   (Procedure, 列舉值 5)
```

如果順序錯了——例如先建立資料表再建立型別——就會得到錯誤：

```
Msg 2715: Column or parameter #3: Cannot find data type dbo.PhoneNumber.
```

透過讓 `DbObjectType` 列舉值本身就蘊含正確的依賴順序，前端 UI 只需要按照列舉值排序，就能自然呈現出安全的複製順序。

> **下一章預告**：模型設計完成後，接下來要設計服務層的介面。第 4 章將討論 `IDbService` 介面的設計——九個方法各自的職責是什麼，為什麼要用介面而不是抽象類別。

4 介面導向設計 — IDbService

4.1 為什麼需要介面

DbCopy 需要同時支援 SQL Server 和 PostgreSQL。這兩個資料庫引擎的語法、系統目錄、大量載入機制都截然不同，但從使用者的角度來看，操作流程是完全相同的：

1. 測試連線
2. 取得物件清單
3. 比較來源與目標
4. 複製物件

介面（Interface）正是為了解決這個問題而存在的。它定義了「做什麼」（What），而不關心「怎麼做」（How）。

```
                    IDbService     ← 定義契約

          ↓                         ↓

SqlServerService          PostgreSqlService
 （用 sys.* 實作）          （用 pg_* 實作）
```

4.1.1 為什麼不用抽象類別？

抽象類別適合「有共通實作」的場景。但 SQL Server 和 PostgreSQL 的查詢語法幾乎沒有任何重疊：

功能	SQL Server	PostgreSQL
系統目錄	`sys.tables`, `sys.columns`	`pg_class`, `pg_attribute`
物件定義	`OBJECT_DEFINITION()`	`pg_get_functiondef()`
大量載入	`SqlBulkCopy`	`COPY (FORMAT BINARY)`
識別符引用	`[name]`	`"name"`
Schema 存在	`sys.schemas`	`information_schema.schemata`

兩者之間**沒有可共用的實作邏輯**，所以介面比抽象類別更適合。

4.2 IDbService 的九個方法

```
public interface IDbService
{
    // 連線管理
    Task<bool> TestConnectionAsync(string connectionString);

    // 物件探索
    Task<List<DbObject>> GetDbObjectsAsync(string connectionString);
    Task<string> GetObjectDefinitionAsync(string connectionString, DbObject obj);
    Task<bool> CheckObjectExistsAsync(string connectionString, DbObject obj);

    // Schema 管理
    Task EnsureSchemaExistsAsync(string connectionString, string schema);

    // 相依性分析
    Task<List<string>> GetDependenciesAsync(string connectionString, DbObject obj);

    // 資料表統計
    Task<long> GetRowCountAsync(string connectionString, DbObject obj);
    Task<List<DbIndex>> GetTableIndexesAsync(string connectionString, DbObject obj);

    // 複製操作
    Task CopyObjectAsync(string sourceConnectionString, string targetConnectionString,
        DbObject obj, int phase = 0, int batchSize = 1000);
}
```

4.3 每個方法的職責與契約

4.3.1 TestConnectionAsync — 連線測試

```
Task<bool> TestConnectionAsync(string connectionString);
```

契約：嘗試開啟連線。成功回傳 `true`；失敗拋出例外（由呼叫端捕捉並顯示錯誤訊息）。

為什麼回傳 `bool` 而不是 `void`？語意上「測試」應該有結果。雖然目前失敗會拋例外，但保留 `bool` 回傳值讓未來可以改為回傳 `false` 而不拋例外。

4.3.2 GetDbObjectsAsync — 物件清單

```
Task<List<DbObject>> GetDbObjectsAsync(string connectionString);
```

契約：回傳資料庫中所有使用者定義的物件，不含系統物件。回傳的 `DbObject` 不包含 `Definition`（延遲載入）。

這是整個系統的起點——所有後續操作都基於這個方法的回傳值。

4.3.3 GetObjectDefinitionAsync — DDL 定義

```
Task<string> GetObjectDefinitionAsync(string connectionString, DbObject obj);
```

契約：回傳物件的完整 DDL 定義字串，可直接在目標資料庫執行。回傳空字串表示無法取得定義。

這是最複雜的方法——不同的物件類型需要完全不同的 DDL 重建策略。

4.3.4 CheckObjectExistsAsync — 存在性檢查

```
Task<bool> CheckObjectExistsAsync(string connectionString, DbObject obj);
```

契約：檢查物件是否已存在於指定的資料庫中。以物件的 Schema + Name + Type 三個維度精確比對。

4.3.5 EnsureSchemaExistsAsync — Schema 建立

```
Task EnsureSchemaExistsAsync(string connectionString, string schema);
```

契約：確保指定的 Schema 存在。若已存在則不做任何事（冪等性）。對於預設 Schema（SQL Server 的 `dbo`、PostgreSQL 的 `public`）直接跳過。

為什麼需要這個方法？來源資料庫可能有 `hr`、`sales` 等自訂 Schema，目標資料庫可能沒有。在建立物件之前必須先確保 Schema 存在。

4.3.6 GetDependenciesAsync — 相依性分析

```
Task<List<string>> GetDependenciesAsync(string connectionString, DbObject obj);
```

契約：回傳此物件所依賴的其他物件的 FullName 清單。

涵蓋範圍：

依賴類型	範例
表達式依賴	預存程序 SELECT FROM 某資料表
欄位型別依賴	資料表的欄位使用自訂型別
參數型別依賴	預存程序的參數使用自訂型別
序列依賴	欄位的預設值使用序列

4.3.7 GetRowCountAsync — 資料表行數

```
Task<long> GetRowCountAsync(string connectionString, DbObject obj);
```

契約：回傳資料表的精確行數。非資料表物件回傳 0。使用 `COUNT_BIG`（SQL Server）或 `COUNT(*)::bigint`（PostgreSQL）以支援超大型資料表。

4.3.8 GetTableIndexesAsync — 索引清單

```
Task<List<DbIndex>> GetTableIndexesAsync(string connectionString, DbObject obj);
```

契約：回傳資料表的所有索引（包含主鍵索引）。每個索引包含名稱、唯一性、鍵欄位列表。

4.3.9 CopyObjectAsync — 複製操作

```
Task CopyObjectAsync(string sourceConnectionString, string targetConnectionString,
    DbObject obj, int phase = 0, int batchSize = 1000);
```

契約：將物件從來源複製到目標。`phase` 參數控制執行哪些階段：

phase	說明
0	全部階段（1□ 2□ 3□ 4）
1	僅結構（CREATE）
2	僅資料（Bulk Copy）

phase	說明
3	僅索引（CREATE INDEX）
4	僅外鍵（ALTER TABLE ADD CONSTRAINT）

這是最核心的方法——它整合了定義取得、結構建立、資料複製、索引建立、外鍵建立等所有操作。

4.4 如何在未來擴展新的資料庫類型

如果要新增 MySQL 支援，只需要三個步驟：

步驟一：擴展 `DbType` 列舉

```
public enum DbType
{
    SqlServer,
    PostgreSql,
    MySql        // 新增
}
```

步驟二：建立 `MySqlService` 實作

```
public class MySqlService(ILogger<MySqlService> logger) : IDbService
{
    // 實作九個方法...
}
```

步驟三：在 `Program.cs` 註冊服務，在 `DbController` 加入分派邏輯

```
// Program.cs
builder.Services.AddScoped<MySqlService>();

// DbController.cs
private IDbService GetService(DbType type) => type switch
{
    DbType.SqlServer => sqlServerService,
    DbType.PostgreSql => postgreSqlService,
    DbType.MySql => mySqlService,        // 新增
    _ => throw new ArgumentException("Unsupported database type")
};
```

不需要修改任何現有的服務實作——這就是介面導向設計的威力。

4.4.1 目前架構的一個務實折衷

你可能注意到，`DbController` 直接注入了具體的 `SqlServerService` 和 `PostgreSqlService`，而不是注入 `IEnumerable<IDbService>` 或使用 Service Locator 模式。

這是一個務實的設計決策。對於只有兩個實作的情境，直接注入比使用工廠模式更簡單、更容易理解。如果未來支援的資料庫引擎增加到 5 個以上，再重構為工廠模式也不遲。

下一章預告：介面定義了「做什麼」，但每一次「做」都伴隨著 SQL 的執行。第 5 章將介紹 DapperExtensions——一個精巧的日誌包裝層，確保每一條 SQL 都被記錄下來。

5 Dapper 擴充方法與結構化日誌

5.1 為什麼要包裝 Dapper

DbCopy 的所有服務層程式碼都會透過 Dapper 執行 SQL 查詢。在偵錯和維運時，有一個非常重要的需求：**我想要在日誌中看到每一條被執行的 SQL**。

如果不做包裝，每個查詢呼叫前都需要手動寫一行日誌：

```
// 不好的做法：每個呼叫前都要記得寫 log
logger.LogInformation("Executing SQL: {Sql}", sql);
var results = await conn.QueryAsync<string>(sql, param);
```

這有三個問題：1. **容易遺漏**：開發者可能忘記加日誌 2. **重複程式碼**：相同的 log 邏輯散布在整個程式碼庫 3. **格式不一致**：不同開發者可能用不同的日誌格式

解決方案：**建立擴充方法，將日誌邏輯封裝在 Dapper 呼叫的外面**。

5.2 DapperExtensions 的設計

完整的 `DapperExtensions.cs`（80 行）提供了五個包裝方法：

```
public static partial class DapperExtensions
{
    // 強型別查詢
    public static Task<IEnumerable<T>> QueryWithLogAsync<T>(
        this IDbConnection conn, ILogger logger, string sql, object? param = null)
    {
        logger.LogExecutingSqlSql(sql);
        return conn.QueryAsync<T>(sql, param);
    }

    // 動態查詢（回傳 dynamic）
    public static Task<IEnumerable<dynamic>> QueryWithLogAsync(
        this IDbConnection conn, ILogger logger, string sql, object? param = null)
    {
        logger.LogExecutingSqlSql(sql);
        return conn.QueryAsync(sql, param);
```

```
    }

    // 單列查詢
    public static Task<dynamic?> QueryFirstOrDefaultWithLogAsync(
        this IDbConnection conn, ILogger logger, string sql, object? param = null)
    {
        logger.LogExecutingSqlSql(sql);
        return conn.QueryFirstOrDefaultAsync(sql, param);
    }

    // 純量查詢
    public static Task<T?> ExecuteScalarWithLogAsync<T>(
        this IDbConnection conn, ILogger logger, string sql, object? param = null)
    {
        logger.LogExecutingSqlSql(sql);
        return conn.ExecuteScalarAsync<T>(sql, param);
    }

    // 非查詢執行 (INSERT/UPDATE/DELETE/DDL)
    public static Task<int> ExecuteWithLogAsync(
        this IDbConnection conn, ILogger logger, string sql, object? param = null)
    {
        logger.LogExecutingSqlSql(sql);
        return conn.ExecuteAsync(sql, param);
    }

    [LoggerMessage(LogLevel.Information, "Executing SQL: {Sql}")]
    static partial void LogExecutingSqlSql(this ILogger logger, string Sql);
}
```

5.2.1 設計要點

擴充方法（Extension Method）

透過 `this IDbConnection conn` 讓所有 `IDbConnection` 物件都能直接呼叫這些方法：

```
// 使用前：直接呼叫 Dapper
await conn.QueryAsync<string>(sql, param);

// 使用後：透過擴充方法呼叫
await conn.QueryWithLogAsync<string>(logger, sql, param);
```

差異只在方法名稱和多一個 `logger` 參數，呼叫端幾乎不需要改變習慣。

額外傳入 `ILogger` 參數

你可能會問：為什麼不在擴充方法內部自己建立 Logger？

答案是 **依賴注入原則**。每個 Service 類別都有自己的 `ILogger<T>`，日誌訊息會標記為來自 `SqlServerService` 或 `PostgreSqlService`。如果在擴充方法內部建立 Logger，所有日誌都會變成來自 `DapperExtensions`，失去了來源追蹤的能力。

不包裝回傳值

擴充方法直接 `return` Dapper 的回傳值，不做任何額外處理。這確保：- 零效能損耗 - 保留 Dapper 的原始行為 - 異常（Exception）自然向上傳播

5.3 LoggerMessage 原始碼產生器

檔案最底部有一行看似普通的程式碼，實際上蘊含了.NET 的高效能日誌技術：

```
[LoggerMessage(LogLevel.Information, "Executing SQL: {Sql}")]
static partial void LogExecutingSqlSql(this ILogger logger, string Sql);
```

5.3.1 為什麼不直接用 `logger.LogInformation()`？

```
// 方式 A：傳統做法
logger.LogInformation("Executing SQL: {Sql}", sql);

// 方式 B：LoggerMessage 原始碼產生器
LogExecutingSqlSql(logger, sql);
```

兩者的功能完全相同，但效能有顯著差異。

方式 A 的問題：1. 每次呼叫都需要解析格式字串 2. 參數需要 boxing（如果是值型別）3. 即使 Information 等級未啟用，仍然會分配字串

方式 B 的優勢：1. 編譯時產生最佳化的程式碼 2. 先檢查日誌等級是否啟用，未啟用時完全跳過 3. 避免不必要的記憶體分配

對於 DbCopy 這種會執行大量 SQL 的工具，這個效能差異是有意義的。

5.3.2 partial class 與 partial method

注意 `DapperExtensions` 類別宣告為 `partial class`，`LogExecutingSqlSql` 方法宣告為 `static partial void`。這是 C# 原始碼產生器（Source Generator）的必要語法：

- **編譯器**看到 `[LoggerMessage]` 屬性，自動在另一個 partial 檔案中產生完整的方法實作
- **開發者**只需要宣告方法簽名和日誌模板

5.4 確保每條 SQL 恰好記錄一次

DapperExtensions 的設計遵循一個嚴格的規則：

> **設計原則：所有 Service 類別必須使用這些 wrapper，不得直接呼叫 Dapper，如此每一次 SQL 往返恰好在日誌中出現一次，不需在呼叫端重複 log。**

這個規則確保了日誌的**完整性**和**無重複性**：

情境	日誌行為
透過 DapperExtensions 呼叫	自動記錄一次
直接呼叫 Dapper	不會記錄（違反規則！）
使用 SqlCommand/NpgsqlCommand	需要手動呼叫 LogExecutingSqlSql

5.4.1 繞過 Dapper 的情況

有些操作無法透過 Dapper 完成，必須直接使用 ADO.NET 的 `SqlCommand` 或 `NpgsqlCommand`：

- **SQL Server 的 SqlBulkCopy**：大量資料載入不走 Dapper
- **PostgreSQL 的 COPY 協定**：二進位資料匯入不走 Dapper
- **SQL Server 的 DBCC CHECKIDENT**：某些系統命令需要直接執行

在這些情況下，Service 類別會定義自己的 `LogExecutingSqlSql` 方法：

```
// SqlServerService.cs
public partial class SqlServerService(ILogger<SqlServerService> logger) : IDbService
{
    // 獨立的 LoggerMessage 定義，用於繞過 Dapper 的程式碼路徑
    [LoggerMessage(LogLevel.Information, "Executing SQL: {Sql}")]
    static partial void LogExecutingSqlSql(ILogger<SqlServerService> logger, string Sql);

    private async Task CopyTableDataAsync(...)
    {
        var sql = $"SELECT * FROM [{obj.Schema}].[{obj.Name}]";
        // 手動記錄 SQL（因為使用 SqlCommand 而非 Dapper）
        LogExecutingSqlSql(logger, sql);
        await using var cmd = new SqlCommand(sql, sourceConn);
        ...
    }
}
```

這種設計讓日誌格式保持一致（`"Executing SQL: {Sql}"`），無論底層是透過 Dapper 還是直接使用 ADO.NET。

5.4.2 日誌輸出範例

實際運行時的日誌輸出：

```
[INF] Executing SQL: SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'Table' AS Type F
[INF] Executing SQL: SELECT 1 FROM sys.tables WHERE SCHEMA_NAME(schema_id) = @Schema AND name =
[INF] Executing SQL: SELECT COUNT_BIG(*) FROM [dbo].[Users]
[INF] Executing SQL: SELECT * FROM [dbo].[Users]
[INF] Executing SQL: DBCC CHECKIDENT ('[dbo].[Users]', RESEED)
```

每一條 SQL 恰好出現一次，方便追蹤問題。

下一章預告：有了 Dapper 包裝層和統一的日誌機制，接下來就要進入最核心的部分——SQL Server 服務的完整實作。第 6 章將逐一講解如何從 `sys.*` 系統目錄中提取物件定義。

6 SQL Server 服務實作

`SqlServerService` 是整個系統中程式碼量最大的服務之一（669 行），因為它需要處理 SQL Server 七種不同物件類型的探索、定義重建、相依性分析、資料複製、索引建立和外鍵建立。

6.1 連線測試

```
public async Task<bool> TestConnectionAsync(string connectionString)
{
    await using var conn = new SqlConnection(connectionString);
    await conn.OpenAsync();
    return true;
}
```

最簡單的方法——嘗試開啟連線。如果連線字串有誤、伺服器不可達、帳密錯誤，`OpenAsync()` 會拋出 `SqlException`。

`await using` 的重要性：確保連線物件在方法結束時被正確釋放（即使發生例外）。

6.2 物件清單查詢 — sys.* 目錄檢視

```
public async Task<List<DbObject>> GetDbObjectsAsync(string connectionString)
{
    await using var conn = new SqlConnection(connectionString);
    const string sql = """
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'Table' AS Type
          FROM sys.tables WHERE is_ms_shipped = 0
        UNION ALL
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'View' AS Type
          FROM sys.views WHERE is_ms_shipped = 0
        UNION ALL
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'Procedure' AS Type
          FROM sys.procedures WHERE is_ms_shipped = 0
        UNION ALL
```

```
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'Function' AS Type
          FROM sys.objects WHERE type IN ('FN', 'IF', 'TF') AND is_ms_shipped = 0
        UNION ALL
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'UserDefinedType' AS Type
          FROM sys.types WHERE is_user_defined = 1 AND is_table_type = 0
        UNION ALL
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'UserDefinedTableType' AS Type
          FROM sys.table_types WHERE is_user_defined = 1
        UNION ALL
        SELECT SCHEMA_NAME(schema_id) AS [Schema], name AS Name, 'Sequence' AS Type
          FROM sys.sequences
        """;

    var results = await conn.QueryWithLogAsync(logger, sql);
    return results.Select(r => new DbObject
    {
        Schema = r.Schema,
        Name = r.Name,
        Type = Enum.Parse<DbObjectType>(r.Type)
    }).ToList();
}
```

6.2.1 SQL Server 系統目錄速覽

系統檢視	內容	過濾條件
sys.tables	使用者資料表	is_ms_shipped = 0 排除系統表
sys.views	檢視	is_ms_shipped = 0 排除系統檢視
sys.procedures	預存程序	is_ms_shipped = 0 排除系統程序
sys.objects	所有物件	type IN ('FN','IF','TF') 篩選函數
sys.types	型別	is_user_defined = 1 篩選自訂型別
sys.table_types	資料表型別	is_user_defined = 1 篩選自訂
sys.sequences	序列	無額外過濾（序列都是使用者定義的）

6.2.2 函數的三種子類型

SQL Server 的函數分為三種：- FN：純量函數（Scalar Function）- IF：行內資料表值函數（Inline Table-valued Function）- TF：多陳述式資料表值函數（Multi-statement Table-valued Function）

在 DbCopy 中，我們將這三種都歸類為 Function 類型，不做細分。

6.2.3 Type 欄位的字串映射

注意 SQL 查詢中 `Type` 欄位使用的是字串(如 `'Table'`, `'View'`)，然後用 `Enum.Parse<DbObjectType>(r.Type)` 轉換。這要求字串值必須與 `DbObjectType` 列舉的成員名稱完全一致。

6.3 DDL 定義重建

`GetObjectDefinitionAsync` 是最複雜的方法，因為不同的物件類型需要完全不同的 DDL 重建策略：

```
public async Task<string> GetObjectDefinitionAsync(string connectionString, DbObject obj)
{
    await using var conn = new SqlConnection(connectionString);
    if (obj.Type == DbObjectType.Table)
        return await GetTableDefinitionAsync(conn, obj);
    if (obj.Type == DbObjectType.UserDefinedType)
        return await GetUserDefinedTypeDefinitionAsync(conn, obj);
    if (obj.Type == DbObjectType.UserDefinedTableType)
        return await GetUserDefinedTableTypeDefinitionAsync(conn, obj);
    if (obj.Type == DbObjectType.Sequence)
        return await GetSequenceDefinitionAsync(conn, obj);

    // 檢視、程序、函數：使用 OBJECT_DEFINITION()
    const string sql =
        "SELECT OBJECT_DEFINITION(OBJECT_ID(QUOTENAME(@Schema) + '.' + QUOTENAME(@Name)))";
    return await conn.ExecuteScalarWithLogAsync<string>(logger, sql,
        new { obj.Schema, obj.Name }) ?? "";
}
```

6.3.1 為什麼有些物件可以用 OBJECT_DEFINITION()，有些不行？

SQL Server 的 `OBJECT_DEFINITION()` 函數可以取得以下物件的原始 SQL 定義文字：- 檢視（View）- 預存程序（Procedure）- 函數（Function）

但以下物件**不被 OBJECT_DEFINITION() 支援**：- 資料表（Table）——因為資料表沒有「定義文字」，它的結構分散在 `sys.columns`、`sys.indexes` 等多個目錄中 - 使用者定義型別（UDT）——型別資訊存在 `sys.types` 中 - 序列（Sequence）——序列資訊存在 `sys.sequences` 中

6.3.2 資料表定義重建

資料表的 DDL 重建是最複雜的，需要查詢三個系統檢視：

```
private async Task<string> GetTableDefinitionAsync(SqlConnection conn, DbObject obj)
{
    // 1. 查詢所有欄位
    var columns = await conn.QueryWithLogAsync(logger, """
        SELECT c.COLUMN_NAME, c.DATA_TYPE, c.CHARACTER_MAXIMUM_LENGTH,
               c.IS_NULLABLE,
               COLUMNPROPERTY(..., 'IsIdentity') AS IsIdentity
        FROM INFORMATION_SCHEMA.COLUMNS c
        WHERE c.TABLE_SCHEMA = @Schema AND c.TABLE_NAME = @Name
        ORDER BY c.ORDINAL_POSITION
        """, new { obj.Schema, obj.Name });

    // 2. 查詢主鍵條件約束名稱
    var pkInfo = await conn.QueryFirstOrDefaultWithLogAsync(logger, """
        SELECT tc.CONSTRAINT_NAME
        FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
        WHERE ... AND tc.CONSTRAINT_TYPE = 'PRIMARY KEY'
        """, new { obj.Schema, obj.Name });

    // 3. 查詢主鍵欄位清單
    var pkColumns = await conn.QueryWithLogAsync<string>(logger, """
        SELECT k.COLUMN_NAME
        FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE k
        ...
        ORDER BY k.ORDINAL_POSITION
        """, new { obj.Schema, obj.Name });

    // 4. 組裝 CREATE TABLE 語句
    return $"CREATE TABLE [{obj.Schema}].[{obj.Name}] (\n  {columnDefs}{pkSql}\n)";
}
```

重點設計：

- 使用 `INFORMATION_SCHEMA` 而非 `sys.columns`，因為 `INFORMATION_SCHEMA.COLUMNS.CHARACTER_MAXIMUM_LE` 已經是字元數，不需要再做位元組換算。
- IDENTITY 欄位透過 `COLUMNPROPERTY()` 函數偵測。
- 主鍵條件約束保留原始名稱（如 `PK_Users`），確保在目標資料庫中保持一致。

6.3.3 序列定義重建

```
private async Task<string> GetSequenceDefinitionAsync(SqlConnection conn, DbObject obj)
{
    var seqInfo = await conn.QueryFirstOrDefaultWithLogAsync(logger, """
```

```
        SELECT t.name AS DataType, s.start_value AS StartValue,
               s.increment AS Increment, s.minimum_value AS MinimumValue,
               s.maximum_value AS MaximumValue, s.is_cycling AS IsCycling
        FROM sys.sequences s
        JOIN sys.types t ON s.user_type_id = t.user_type_id
        WHERE SCHEMA_NAME(s.schema_id) = @Schema AND s.name = @Name
        """, new { obj.Schema, obj.Name });

    return $"CREATE SEQUENCE [{obj.Schema}].[{obj.Name}] " +
           $"AS [{seqInfo.DataType}] " +
           $"START WITH {seqInfo.StartValue} " +
           $"INCREMENT BY {seqInfo.Increment} " +
           $"MINVALUE {seqInfo.MinimumValue} " +
           $"MAXVALUE {seqInfo.MaximumValue} " +
           (seqInfo.IsCycling ? "CYCLE" : "NO CYCLE") + ";";
}
```

6.4 型別長度的位元組與字元換算

SQL Server 的系統目錄中，`max_length` 是以**位元組**為單位存儲的。這在處理 Unicode 型別時會造成混淆：

型別	DDL 定義	sys.columns.max_length
`varchar(50)`	50 字元	50 位元組
`nvarchar(50)`	50 字元	**100 位元組**
`char(10)`	10 字元	10 位元組
`nchar(10)`	10 字元	**20 位元組**
`varbinary(100)`	100 位元組	100 位元組
`varchar(MAX)`	不限	**-1**

`FormatSqlServerTypeLength` 方法處理這個換算：

```
private static readonly string[] TypesWithByteLength =
    ["varchar", "nvarchar", "char", "nchar", "varbinary", "binary"];

private static string FormatSqlServerTypeLength(string typeName, int maxLengthBytes)
{
    if (!TypesWithByteLength.Contains(typeName.ToLower())) return "";
    var charLength = typeName.StartsWith("n", StringComparison.OrdinalIgnoreCase)
        ? maxLengthBytes / 2    // Unicode：除以 2
        : maxLengthBytes;       // 非 Unicode：直接使用
    return charLength == -1 ? "(MAX)" : $"({charLength})";
}
```

6.4.1 注意：兩個不同的長度來源

在 DbCopy 中，長度資訊來自兩個不同的地方：

1. `sys.columns.max_length`（位元組）——用於 `GetUserDefinedTypeDefinitionAsync` 和 `GetUserDefinedTableTypeDefinitionAsync`
2. `INFORMATION_SCHEMA.COLUMNS.CHARACTER_MAXIMUM_LENGTH`(字元數)——用於 `GetTableDefinitionAsync`

使用 `INFORMATION_SCHEMA` 時**不需要**除以 2，因為它已經是字元數。使用 `sys.columns` 時才需要透過 `FormatSqlServerTypeLength` 換算。

6.5 相依性分析

```
public async Task<List<string>> GetDependenciesAsync(string connectionString, DbObject obj)
{
    await using var conn = new SqlConnection(connectionString);

    // 1. 表達式層級的依賴（FROM, JOIN, EXEC 等）
    const string sql = """
        SELECT DISTINCT
            COALESCE(referenced_schema_name, SCHEMA_NAME(o.schema_id), 'dbo')
            + '.' + referenced_entity_name
        FROM sys.sql_expression_dependencies sed
        LEFT JOIN sys.objects o ON sed.referenced_id = o.object_id
        WHERE referencing_id = OBJECT_ID(QUOTENAME(@Schema) + '.' + QUOTENAME(@Name))
          AND referenced_entity_name IS NOT NULL
        """;
    var results = (await conn.QueryWithLogAsync<string>(logger, sql, ...)).ToList();

    // 2. 欄位型別的 UDT 依賴
    const string udtSql = """
        SELECT DISTINCT SCHEMA_NAME(t.schema_id) + '.' + t.name
        FROM sys.columns c
        JOIN sys.types t ON c.user_type_id = t.user_type_id
        WHERE c.object_id = OBJECT_ID(...) AND t.is_user_defined = 1
        """;
    results.AddRange(await conn.QueryWithLogAsync<string>(logger, udtSql, ...));

    // 3. 參數型別的 UDT/UDTT 依賴（僅程序和函數）
    if (obj.Type is DbObjectType.Procedure or DbObjectType.Function)
    {
        const string paramSql = """
            SELECT DISTINCT SCHEMA_NAME(t.schema_id) + '.' + t.name
```

```
            FROM sys.parameters p
            JOIN sys.types t ON p.user_type_id = t.user_type_id
            WHERE p.object_id = OBJECT_ID(...) AND t.is_user_defined = 1
            """;
        results.AddRange(await conn.QueryWithLogAsync<string>(logger, paramSql, ...));
    }

    return results.Distinct().ToList();
}
```

6.5.1 依賴分析的三個維度

1. **表達式依賴**（sys.sql_expression_dependencies）：記錄了 SQL 程式碼中引用的所有物件。例如，檢視 SELECT * FROM Users 會記錄對 Users 的依賴。
2. **欄位型別依賴**（sys.columns + sys.types）：如果資料表的某個欄位型別是使用者定義型別，sys.sql_expression_dependencies 不會記錄這種依賴，需要額外查詢。
3. **參數型別依賴**（sys.parameters + sys.types）：預存程序的參數如果使用了使用者定義型別或資料表型別，也需要額外查詢。

6.6 SqlBulkCopy 大量資料複製

```
private async Task CopyTableDataAsync(string sourceConnectionString,
    string targetConnectionString, DbObject obj, int batchSize)
{
    await using var sourceConn = new SqlConnection(sourceConnectionString);
    await sourceConn.OpenAsync();

    var sql = $"SELECT * FROM [{obj.Schema.Replace("]", "]]")}].[{obj.Name.Replace("]", "]]")}]
    await using var cmd = new SqlCommand(sql, sourceConn);
    LogExecutingSqlSql(logger, sql);  // 手動記錄 SQL
    await using var reader = await cmd.ExecuteReaderAsync();

    await using var targetConn = new SqlConnection(targetConnectionString);
    await targetConn.OpenAsync();

    using var bulkCopy = new SqlBulkCopy(targetConn,
        SqlBulkCopyOptions.KeepIdentity | SqlBulkCopyOptions.KeepNulls, null);
    bulkCopy.DestinationTableName =
        $"[{obj.Schema.Replace("]", "]]")}].[{obj.Name.Replace("]", "]]")}]";
    bulkCopy.BulkCopyTimeout = 600;  // 10 分鐘
```

```
    bulkCopy.BatchSize = batchSize;

    await bulkCopy.WriteToServerAsync(reader);
    ...
}
```

6.6.1 SqlBulkCopy 的關鍵選項

選項	說明
`KeepIdentity`	保留來源的 IDENTITY 值，不讓目標自動產生新值
`KeepNulls`	保留 NULL 值，不使用目標欄位的預設值替代
`BulkCopyTimeout = 600`	10 分鐘逾時（大型資料表需要較長時間）
`BatchSize`	每批次寫入的列數（預設 1000）

6.6.2 串流模式

注意資料複製使用的是**串流模式**：`SqlDataReader` 逐列讀取來源資料，`SqlBulkCopy` 逐批次寫入目標。整個過程不需要將所有資料載入記憶體，因此即使是數百萬列的大型資料表也不會造成記憶體不足。

6.7 IDENTITY 欄位重設

```
// 檢查是否有 IDENTITY 欄位
var hasIdentity = await targetConn.ExecuteScalarWithLogAsync<int?>(logger,
    "SELECT 1 FROM sys.identity_columns WHERE object_id = OBJECT_ID(...)",
    new { obj.Schema, obj.Name }) != null;

if (hasIdentity)
{
    var reseedSql =
        $"DBCC CHECKIDENT ('[{obj.Schema}].[{obj.Name}]', RESEED)";
    LogExecutingSqlSql(logger, reseedSql);
    await using var reseedCmd = new SqlCommand(reseedSql, targetConn);
    await reseedCmd.ExecuteNonQueryAsync();
}
```

6.7.1 為什麼需要重設 IDENTITY？

當使用 `KeepIdentity` 選項複製資料時，目標資料表的 IDENTITY 種子值不會自動更新。如果最後一筆資料的 ID 是 1000，但 IDENTITY 種子值仍然是 1，後續的 INSERT 就會嘗試插入 ID=1，導致主鍵衝突。

`DBCC CHECKIDENT(table, RESEED)` 會將種子值重設為資料表中 IDENTITY 欄位的最大值。

6.8 索引與外鍵的建立

6.8.1 索引定義提取

```
private async Task<List<string>> GetTableIndexDefinitionsAsync(...)
{
    var rows = await conn.QueryWithLogAsync(logger, """
        SELECT i.name AS IndexName, i.is_unique AS IsUnique,
               c.name AS ColumnName, ic.is_included_column AS IsIncluded,
               ic.key_ordinal AS KeyOrdinal
        FROM sys.indexes i
        JOIN sys.index_columns ic ON ...
        JOIN sys.columns c ON ...
        WHERE i.object_id = OBJECT_ID(...) AND i.type > 0 AND i.is_primary_key = 0
        ORDER BY i.name, ic.is_included_column, ic.key_ordinal
        """, new { obj.Schema, obj.Name });

    // 以索引名稱分組，區分 key columns 和 include columns
    var grouped = rows.GroupBy(r => new { Name = r.IndexName, IsUnique = r.IsUnique });

    foreach (var g in grouped)
    {
        var keyCols = g.Where(r => !r.IsIncluded).Select(r => $"[{r.ColumnName}]");
        var incCols = g.Where(r => r.IsIncluded).Select(r => $"[{r.ColumnName}]");
        var includeSql = incCols.Any() ? $" INCLUDE ({string.Join(", ", incCols)})" : "";
        sqls.Add($"CREATE {(g.Key.IsUnique ? "UNIQUE " : "")}INDEX [{g.Key.Name}] " +
                 $"ON [{obj.Schema}].[{obj.Name}] ({string.Join(", ", keyCols)}){includeSql}");
    }
}
```

關鍵設計：- `i.type > 0` 排除 Heap 虛擬索引 - `i.is_primary_key = 0` 排除主鍵索引（主鍵已在 Phase 1 的 CREATE TABLE 中建立）- `is_included_column` 區分索引鍵欄和 INCLUDE 欄

6.8.2 外鍵定義提取

外鍵的提取邏輯類似，但需要額外處理多欄位外鍵的情況：

```
// 以外鍵名稱分組
var grouped = fks.GroupBy(f => (string)f.ForeignKeyName);

foreach (var g in grouped)
{
    var cols = string.Join(", ", g.Select(r => $"[{r.ColumnName}]"));
    var refCols = string.Join(", ", g.Select(r => $"[{r.ReferencedColumnName}]"));
    sqls.Add($"ALTER TABLE [{first.SchemaName}].[{first.TableName}] " +
             $"ADD CONSTRAINT [{g.Key}] FOREIGN KEY ({cols}) " +
             $"REFERENCES [{first.ReferencedSchemaName}].[{first.ReferencedTableName}] ({refCol
}
```

6.8.3 容錯設計

索引和外鍵的建立採用「盡力而為」策略：

```
// 索引：失敗記錄警告，繼續下一個
catch (Exception ex)
{
    logger.LogWarning(ex, "Failed to create index for {Schema}.{Name}", ...);
}

// 外鍵：特別處理「參考物件不存在」的錯誤
catch (SqlException ex) when (ex.Number == 4902)
{
    logger.LogInformation("Skipped foreign key - referenced object does not exist (Error 4902)'
}
```

Error 4902 是一個特別常見的情況：當複製部分資料表時，外鍵參考的資料表可能尚未存在於目標資料庫中。這不是一個錯誤，而是預期的行為。

下一章預告：同樣的九個方法，在 PostgreSQL 上的實作完全不同。第 7 章將介紹 PostgreSQL 服務——從 pg_catalog 目錄查詢到 COPY 協定的大量資料載入。

7 PostgreSQL 服務實作

`PostgreSqlService` 是整個系統中最龐大的服務（約 943 行），因為 PostgreSQL 的系統目錄結構與 SQL Server 截然不同，且 PostgreSQL 的物件類型更加多元（Enum、Composite、Domain 型別、分區表、物化檢視等）。

7.1 pg_catalog 與 information_schema 的選擇

PostgreSQL 提供兩套查詢資料庫中繼資料的方式：

方式	優點	缺點
`information_schema`	SQL 標準、可移植性高	資訊有限、不含 PostgreSQL 特有功能
`pg_catalog`	完整資訊、包含所有細節	PostgreSQL 專有、欄位名稱較不直觀

DbCopy 的策略是：**核心查詢使用 `pg_catalog`，輔助查詢視需求選用**。

查詢需求	使用	原因
物件清單	`pg_catalog`	需要 `relkind` 區分分區表/物化檢視
欄位資訊	`pg_catalog` (`pg_attribute`)	需要 `format_type()` 取得完整型別字串
主鍵資訊	`information_schema`	語法較簡單，效能差異不大
序列資訊	`information_schema`	提供了更簡單的查詢方式
外鍵資訊	`information_schema`	查詢語法較直覺
物件定義	`pg_catalog`	需要 `pg_get_functiondef()` 等函數

7.2 排除 Extension 擁有的物件

PostgreSQL 有一個獨特的概念：**Extension**（擴展）。安裝 Extension（如 `postgis`、`uuid-ossp`）會在資料庫中建立物件（函數、型別、運算子等）。這些物件由 Extension 管理，**不應該被複製**。

```
-- 排除屬於 Extension 的物件
AND NOT EXISTS (
    SELECT 1
    FROM pg_depend d
    JOIN pg_extension e ON e.oid = d.refobjid
    WHERE d.objid = c.oid AND d.deptype = 'e'
)
```

`pg_depend.deptype = 'e'` 表示「Extension 擁有」。透過這個過濾條件，DbCopy 可以正確地：- 列出 `postgis` Extension 建立的 `geometry` 型別 □ **排除**（由 Extension 管理）- 列出使用者自己建立的 `PhoneNumber` 型別 □ **包含** - 列出使用 `geometry` 型別的資料表 □ **包含**（資料表本身不是 Extension 物件）

7.3 使用者定義型別的四種子類型

PostgreSQL 的使用者定義型別比 SQL Server 豐富得多，分為四種子類型：

7.3.1 Enum 型別

```
-- PostgreSQL
CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');
```

DbCopy 從 `pg_enum` 表取得列舉值：

```
var enumValues = await conn.QueryWithLogAsync<string>(logger, """
    SELECT e.enumlabel
    FROM pg_type t
    JOIN pg_enum e ON t.oid = e.enumtypid
    JOIN pg_namespace n ON n.oid = t.typnamespace
    WHERE t.typname = @Name AND n.nspname = @Schema
    ORDER BY e.enumsortorder
    """, new { obj.Name, obj.Schema });

if (enumList.Count != 0)
    return $"CREATE TYPE \"{obj.Schema}\".\"{obj.Name}\" AS ENUM " +
           $"('{string.Join("', '", enumList)}')";
```

注意 `ORDER BY e.enumsortorder`——Enum 值的順序很重要，因為 PostgreSQL 的 Enum 是有序的，可以做大小比較。

7.3.2 Composite 型別（複合型別）

```
-- PostgreSQL
CREATE TYPE address AS (
    street TEXT,
    city TEXT,
    zip_code VARCHAR(10)
);
```

```
var compositeCols = await conn.QueryWithLogAsync(logger, """
    SELECT a.attname as Name, format_type(a.atttypid, a.atttypmod) as DataType
    FROM pg_type t
    JOIN pg_class c ON t.typrelid = c.oid
    JOIN pg_attribute a ON a.attrelid = c.oid
    JOIN pg_namespace n ON n.oid = t.typnamespace
    WHERE t.typname = @Name AND n.nspname = @Schema
      AND a.attnum > 0 AND NOT a.attisdropped
    ORDER BY a.attnum
    """, new { obj.Name, obj.Schema });
```

`format_type(a.atttypid, a.atttypmod)` 是 PostgreSQL 提供的神器函數——它會自動將型別 OID 和修飾符轉換為完整的型別字串（如 `character varying(100)`、`numeric(10,2)`），省去了我們手動拼湊的麻煩。

7.3.3 Domain 型別

```
-- PostgreSQL
CREATE DOMAIN positive_integer AS integer CHECK (VALUE > 0);
```

```
var domainInfo = await conn.QueryFirstOrDefaultWithLogAsync(logger, """
    SELECT format_type(typbasetype, typtypmod) as BaseType,
           typnotnull, typdefault
    FROM pg_type t
    JOIN pg_namespace n ON n.oid = t.typnamespace
    WHERE t.typname = @Name AND n.nspname = @Schema AND t.typtype = 'd'
    """, new { obj.Name, obj.Schema });
```

7.3.4 Base 型別（Shell 定義）

完整的 Base 型別需要 C 語言函數，DbCopy 僅產生 Shell 宣告：

```
return $"CREATE TYPE \"{obj.Schema}\".\"{obj.Name}\"";
```

7.3.5 自動偵測型別子類型

DbCopy 透過**逐一嘗試**的方式偵測型別子類型，而非事先查詢 typtype。程式碼邏輯：

```
嘗試 Enum 查詢 → 有結果？→ 回傳 Enum DDL
   ↓ 無結果
嘗試 Composite 查詢 → 有結果？→ 回傳 Composite DDL
   ↓ 無結果
嘗試 Domain 查詢 → 有結果？→ 回傳 Domain DDL
   ↓ 無結果
嘗試 Base 查詢 → 有結果？→ 回傳 Shell DDL
   ↓ 無結果
回傳空字串
```

7.4 COPY 協定：二進位模式與文字模式

PostgreSQL 提供了 COPY 命令用於大量資料匯入匯出，效率遠高於逐列 INSERT。Npgsql（.NET 的 PostgreSQL 驅動程式）提供了兩種模式：

7.4.1 二進位模式（Binary Format）

```
await using var importer = await targetConn.BeginBinaryImportAsync(
    $"COPY \"{obj.Schema}\".\"{obj.Name}\" ({colList}) FROM STDIN (FORMAT BINARY)");

while (await reader.ReadAsync())
{
    await importer.StartRowAsync();
    for (var i = 0; i < reader.FieldCount; i++)
        await importer.WriteAsync(reader.GetValue(i));
}
await importer.CompleteAsync();
```

優點：效能最高，型別對應由 Npgsql 自動處理 **缺點**：不支援某些特殊型別（如空間資料 geometry）

7.4.2 文字模式（Text Format）

```
await using var writer = await targetConn.BeginTextImportAsync(
    $"COPY \"{obj.Schema}\".\"{obj.Name}\" ({colList}) FROM STDIN");

while (await reader.ReadAsync())
{
    var values = new string[reader.FieldCount];
    for (var i = 0; i < reader.FieldCount; i++)
    {
        var val = reader.GetValue(i);
        if (val == DBNull.Value)
            values[i] = "\\N";           // NULL 表示法
        else if (val is byte[] bytes)
            values[i] = "\\\\x" + BitConverter.ToString(bytes).Replace("-", "");
        else if (val is bool b)
            values[i] = b ? "t" : "f";   // boolean 表示法
        else
        {
            var s = val.ToString() ?? "";
            values[i] = s.Replace("\\", "\\\\")   // 跳脫反斜線
                         .Replace("\t", "\\t")     // 跳脫 Tab
                         .Replace("\n", "\\n")     // 跳脫換行
                         .Replace("\r", "\\r");    // 跳脫回車
        }
    }
    await writer.WriteLineAsync(string.Join("\t", values));
}
```

優點：支援所有型別，包括空間資料 **缺點**：需要手動處理跳脫字元

7.4.3 模式選擇邏輯

```
var hasGeometry = columns.Any(c =>
    c.data_type.Contains("geometry", ...) ||
    c.data_type.Contains("geography", ...) ||
    c.data_type.Contains("raster", ...) ||
    c.data_type.Contains("USER-DEFINED", ...));

if (hasGeometry)
    // 使用文字模式，以 ST_AsEWKT() 序列化空間資料
else
    // 使用二進位模式（效率更高）
```

7.5 空間資料的特殊處理

當資料表包含空間欄位（geometry、geography、raster）時，DbCopy 會：

1. **讀取時**：使用 ST_AsEWKT() 將空間資料轉換為 EWKT 文字格式
2. **寫入時**：透過文字模式 COPY，讓 PostgreSQL 自動從 EWKT 解析

```
var selectListForRead = string.Join(", ", columns.Select(c =>
{
    bool isSpatial = dt.Contains("geometry", ...) || ...;
    return isSpatial
        ? $"ST_AsEWKT(\"{c.column_name}\") AS \"{c.column_name}\""  // 轉為文字
        : $"\"{c.column_name}\"";                                    // 原樣讀取
}));
```

7.6 Extension 自動偵測與安裝

EnsureExtensionsIfUsedAsync 是 PostgreSQL 服務獨有的方法。它會分析來源資料表使用的型別和索引，自動在目標資料庫安裝所需的 Extension：

```
private async Task EnsureExtensionsIfUsedAsync(NpgsqlConnection targetConn,
    string sourceConnectionString, DbObject obj)
{
    var extsToEnsure = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

    // 1. 欄位型別偵測
    // geometry/geography/raster → postgis
    // citext → citext
    // hstore → hstore

    // 2. 預設值偵測
    // uuid_generate_* → uuid-ossp

    // 3. 索引運算子類別偵測
    // gin_trgm_ops/gist_trgm_ops → pg_trgm

    foreach (var ext in extsToEnsure)
    {
        var exists = await targetConn.ExecuteScalarWithLogAsync<bool?>(logger,
            "SELECT EXISTS (SELECT 1 FROM pg_extension WHERE extname = @ext)",
            new { ext }) ?? false;
        if (!exists)
        {
```

```
            await targetConn.ExecuteWithLogAsync(logger,
                $"CREATE EXTENSION IF NOT EXISTS \"{ext}\"");
        }
    }
}
```

7.6.1 偵測規則

偵測條件	需要的 Extension
欄位型別含 geometry / geography / raster	postgis
欄位型別為 citext	citext
欄位型別為 hstore	hstore
欄位預設值含 uuid_generate_*	uuid-ossp
索引使用 gin_trgm_ops / gist_trgm_ops	pg_trgm

7.7 序列值重設

PostgreSQL 的序列（Sequence）等同於 SQL Server 的 IDENTITY。資料複製完成後需要重設序列值：

```
// 查詢與此資料表欄位關聯的所有序列
const string seqSql = """
    SELECT s.relname AS SequenceName, n.nspname AS SequenceSchema,
           a.attname AS ColumnName
    FROM pg_class s
    JOIN pg_depend d ON d.objid = s.oid
    JOIN pg_class t ON d.refobjid = t.oid
    JOIN pg_namespace n ON n.oid = s.relnamespace
    JOIN pg_namespace tn ON tn.oid = t.relnamespace
    JOIN pg_attribute a ON a.attrelid = t.oid AND a.attnum = d.refobjsubid
    WHERE s.relkind = 'S' AND t.relname = @Name AND tn.nspname = @Schema
    """;

foreach (var seq in sequences)
{
    var setValSql = $"""
        DO $$
        DECLARE max_val bigint;
        BEGIN
            EXECUTE format('SELECT MAX(%I) FROM %I.%I',
                '{escapedCol}', '{escapedSchema}', '{escapedName}') INTO max_val;
```

```
            IF max_val IS NULL THEN
                PERFORM setval(format('%I.%I', '{escapedSeqSchema}', '{escapedSeqName}'),
                    1, false);
            ELSE
                PERFORM setval(format('%I.%I', '{escapedSeqSchema}', '{escapedSeqName}'),
                    max_val, true);
            END IF;
        END $$;
        """;
    await targetConn.ExecuteWithLogAsync(logger, setValSql);
}
```

7.7.1 設計要點

使用 PL/pgSQL 匿名區塊（`DO $$...$$`）

為什麼不直接用 `SELECT setval(..., (SELECT MAX(...)))`？因為當資料表為空時，`MAX()` 回傳 NULL，`setval(seq, NULL)` 會拋出錯誤。使用匿名區塊可以先判斷是否為 NULL，空表時使用 `setval(seq, 1, false)`（重設為起始值但尚未使用）。

`format('%I', ...)` 防止注入

在 PL/pgSQL 中，`format('%I', identifier)` 等同於 `QUOTENAME()` / `"identifier"`，會自動為識別符加上雙引號並處理跳脫。

`EscapePgLiteral()` 處理字串常值中的單引號

```
private static string EscapePgLiteral(string s) => s.Replace("'", "''");
```

傳入 `format()` 的參數是字串常值，需要跳脫單引號。

下一章預告：服務層的兩大實作已經完成。第 8 章將介紹 API 控制器層——如何將服務層的功能暴露為 RESTful API，以及平行化查詢的策略。

8 RESTful API 設計 — DbController

8.1 API 端點規劃

DbCopy 的 API 層只有一個控制器 `DbController`，提供四個端點：

方法	路徑	功能	輸入	輸出
POST	`/api/db/test`	測試連線	`DbConnectionInfo`	`{ success: true }`
POST	`/api/db/objects`	列出物件	`DbConnectionInfo`	`DbObject[]`
POST	`/api/db/compare`	比較差異	`CompareRequest`	`SyncStatus[]`
POST	`/api/db/copy`	複製物件	`CopyRequest`	`{ success: true }`

8.1.1 為什麼全部用 POST？

傳統的 RESTful 設計會用 GET 讀取、POST 建立。但 DbCopy 的所有請求都包含連線字串（敏感資訊），不適合放在 URL 參數中。因此統一使用 POST，將資料放在 Request Body 中。

8.1.2 路由配置

```
[ApiController]
[Route("api/[controller]")]
public class DbController(...) : ControllerBase
```

- `[ApiController]`：啟用 API 行為（自動模型驗證、自動 `[FromBody]` 綁定）
- `[Route("api/[controller]")]`：`[controller]` 會被替換為類別名稱 `Db`，生成路由前綴 `api/db`

8.2 Primary Constructor 注入

DbCopy 使用 C# 12 的 **Primary Constructor** 語法進行依賴注入：

```
public class DbController(
    SqlServerService sqlServerService,
    PostgreSqlService postgreSqlService,
    ILogger<DbController> logger)
    : ControllerBase
{
    private IDbService GetService(DbType type) => type switch
    {
        DbType.SqlServer => sqlServerService,
        DbType.PostgreSql => postgreSqlService,
        _ => throw new ArgumentException("Unsupported database type")
    };
    ...
}
```

8.2.1 Primary Constructor 的優勢

傳統寫法需要宣告欄位、撰寫建構子、賦值：

```
// 傳統寫法（冗長）
public class DbController : ControllerBase
{
    private readonly SqlServerService _sqlServerService;
    private readonly PostgreSqlService _postgreSqlService;
    private readonly ILogger<DbController> _logger;

    public DbController(SqlServerService sqlServerService,
        PostgreSqlService postgreSqlService, ILogger<DbController> logger)
    {
        _sqlServerService = sqlServerService;
        _postgreSqlService = postgreSqlService;
        _logger = logger;
    }
}
```

Primary Constructor 將以上程式碼壓縮成一行，參數自動成為整個類別可用的變數。

8.2.2 GetService — 服務分派

```
private IDbService GetService(DbType type) => type switch
{
    DbType.SqlServer => sqlServerService,
    DbType.PostgreSql => postgreSqlService,
    _ => throw new ArgumentException("Unsupported database type")
};
```

這個私有方法是 API 層與服務層之間的**橋梁**。根據請求中的 `DbType`，選擇對應的服務實作。上層程式碼完全不需要知道具體用的是哪個服務。

8.3 四大端點的實作

8.3.1 test — 連線測試

```
[HttpPost("test")]
public async Task<IActionResult> TestConnection([FromBody] DbConnectionInfo connection)
{
    try
    {
        var service = GetService(connection.Type);
        var success = await service.TestConnectionAsync(connection.ConnectionString);
        return Ok(new { success });
    }
    catch (Exception ex)
    {
        logger.LogError(ex, "TestConnection failed");
        return BadRequest(ex.Message);
    }
}
```

最簡單的端點——委派給服務層測試連線，成功回傳 `{ success: true }`，失敗回傳錯誤訊息。

8.3.2 objects — 列出物件

```
[HttpPost("objects")]
public async Task<IActionResult> GetObjects([FromBody] DbConnectionInfo connection)
{
    var service = GetService(connection.Type);
    var objects = await service.GetDbObjectsAsync(connection.ConnectionString);
    return Ok(objects);
}
```

同樣是簡單的委派。回傳的 `DbObject[]` 會被 ASP.NET Core 自動序列化為 JSON。

8.3.3 compare — 比較差異（核心端點）

```
[HttpPost("compare")]
public async Task<IActionResult> Compare([FromBody] CompareRequest request)
{
    var sourceService = GetService(request.Source.Type);
    var targetService = GetService(request.Target.Type);

    var sourceObjects = await sourceService.GetDbObjectsAsync(request.Source.ConnectionString);
    var syncStatuses = new List<SyncStatus>();

    foreach (var obj in sourceObjects)
    {
        // 獨立查詢平行化
        var existsTask = targetService.CheckObjectExistsAsync(...);
        var depsTask = sourceService.GetDependenciesAsync(...);
        await Task.WhenAll(existsTask, depsTask);

        if (obj.Type == DbObjectType.Table)
        {
            // 四個資料表查詢也平行化
            var sourceRowsTask = SafeGetRowCount(sourceService, ...);
            var sourceIndexesTask = SafeGetIndexes(sourceService, ...);
            var targetRowsTask = exists ? SafeGetRowCount(targetService, ...) : Task.FromResult
            var targetIndexesTask = exists ? SafeGetIndexes(targetService, ...) : Task.FromResu
            await Task.WhenAll(sourceRowsTask, sourceIndexesTask, targetRowsTask, targetIndexes
            ...
        }

        syncStatuses.Add(new SyncStatus { ... });
    }
```

```
    return Ok(syncStatuses);
}
```

8.3.4 copy — 複製物件

```
[HttpPost("copy")]
public async Task<IActionResult> Copy([FromBody] CopyRequest request)
{
    var sourceService = GetService(request.Source.Type);
    var targetService = GetService(request.Target.Type);

    // 1. 確保 Schema 存在
    await targetService.EnsureSchemaExistsAsync(request.Target.ConnectionString,
        request.Object.Schema);

    // 2. 檢查物件是否已存在（僅 Phase 0 或 1）
    if (request.Phase is 0 or 1)
    {
        var exists = await targetService.CheckObjectExistsAsync(...);
        if (exists) return BadRequest("Object already exists in destination.");
    }

    // 3. 執行複製
    await sourceService.CopyObjectAsync(
        request.Source.ConnectionString,
        request.Target.ConnectionString,
        request.Object, request.Phase, request.BatchSize);

    return Ok(new { success = true });
}
```

設計要點：

- **Schema 先行**：在嘗試建立任何物件之前，先確保目標 Schema 存在
- **存在性檢查有條件**：只在 Phase 0（全部）或 Phase 1（結構）時檢查。Phase 2（資料）、3（索引）、4（外鍵）不需要——它們假設結構已經存在
- **CopyObjectAsync 的來源服務**：注意複製操作是由**來源服務**的 `CopyObjectAsync` 負責，因為它需要讀取來源的資料和定義

8.4 平行化查詢策略

`Compare` 端點中大量使用了 `Task.WhenAll` 進行平行化查詢。這是 DbCopy 效能的關鍵。

8.4.1 第一層平行化：存在性檢查 + 相依性分析

```
// 這兩個查詢完全獨立——一個查來源、一個查目標
var existsTask = targetService.CheckObjectExistsAsync(request.Target.ConnectionString, obj);
var depsTask = sourceService.GetDependenciesAsync(request.Source.ConnectionString, obj);
await Task.WhenAll(existsTask, depsTask);
```

為什麼可以平行？因為它們查的是不同的資料庫（一個來源、一個目標），完全沒有相依性。

8.4.2 第二層平行化：資料表的四個查詢

```
var sourceRowsTask = SafeGetRowCount(sourceService, request.Source.ConnectionString, obj, "sour
var sourceIndexesTask = SafeGetIndexes(sourceService, request.Source.ConnectionString, obj, "so
var targetRowsTask = exists
    ? SafeGetRowCount(targetService, request.Target.ConnectionString, obj, "target")
    : Task.FromResult<long?>(null);
var targetIndexesTask = exists
    ? SafeGetIndexes(targetService, request.Target.ConnectionString, obj, "target")
    : Task.FromResult(new List<DbIndex>());

await Task.WhenAll(sourceRowsTask, sourceIndexesTask, targetRowsTask, targetIndexesTask);
```

四個獨立的查詢同時發出，等待全部完成。如果目標物件不存在，直接用 `Task.FromResult` 回傳預設值，避免不必要的查詢。

8.4.3 Safe Wrapper 模式

```
async Task<long?> SafeGetRowCount(IDbService svc, string cs, DbObject o, string side)
{
    try { return await svc.GetRowCountAsync(cs, o); }
    catch (Exception ex)
    {
        logger.LogWarning(ex, "Failed to get {Side} row count for {Schema}.{Name}",
            side, o.Schema, o.Name);
        return null;
    }
}
```

`Task.WhenAll` 的問題是：如果任何一個 Task 失敗，整個 `await` 就會拋出例外。但行數查詢和索引查詢的失敗**不應該影響**比較結果的回傳。

透過 `SafeGetRowCount` 和 `SafeGetIndexes` 包裝，個別查詢的失敗會被捕捉並記錄為警告，而不是中斷整個比較流程。

8.5 錯誤處理與回應格式

所有端點都遵循統一的錯誤處理模式：

```
try
{
    // 業務邏輯
    return Ok(result);          // 成功：200 OK
}
catch (Exception ex)
{
    logger.LogError(ex, " 操作名稱 failed");
    return BadRequest(ex.Message);  // 失敗：400 Bad Request
}
```

8.5.1 回應格式

場景	HTTP 狀態碼	回應內容
連線測試成功	200	`{ "success": true }`
比較成功	200	`SyncStatus[]` (JSON 陣列)
複製成功	200	`{ "success": true }`
物件已存在	400	`"Object already exists in destination."`
連線失敗	400	例外訊息（純文字）
複製失敗	400	例外訊息（純文字）

設計選擇：使用 `BadRequest` 而非 `500 Internal Server Error`，因為大多數錯誤源自使用者輸入（錯誤的連線字串、目標物件已存在等），而非伺服器內部錯誤。

下一章預告：API 層完成後，使用者需要一個友善的介面來操作。第 9 章將介紹 DbCopy 的單頁式 Web 介面——從連線管理到樹狀結果表格的完整設計。

9 單頁式 Web 介面

DbCopy 的整個 UI 集中在一個 Razor Page（`Index.cshtml`，約 800 行）中。它結合了 Razor 伺服端渲染和 jQuery 前端互動，實現了一個功能完整的單頁應用。

9.1 Razor Pages 與前端 JavaScript 的協作

DbCopy 採用的是 **Razor Pages 負責結構、JavaScript 負責行為**的協作模式：

職責	技術	範例
HTML 結構	Razor	工具列、表格骨架、Modal 對話框
動態資料	jQuery + Fetch API	載入物件清單、更新狀態
狀態管理	localStorage	儲存連線設定
使用者互動	jQuery 事件處理	按鈕點擊、篩選、全選

9.1.1 為什麼不用 Razor 的 OnPost Handler？

Razor Pages 提供了 `OnPostAsync` 等伺服端處理方法，但 DbCopy 完全不使用它們。原因是：

1. **即時回饋**：使用者需要看到即時的進度更新（進度條、狀態變更），Form Post 會導致頁面重新載入
2. **批次操作**：同步操作需要逐一發送請求並更新 UI，不適合 Form Post
3. **狀態保持**：使用 JavaScript 管理頁面狀態，比 Razor 的 ViewData 更靈活

9.2 連線管理系統

連線管理是 DbCopy 的第一道門——使用者必須先設定好連線，才能進行比較和同步。

9.2.1 資料結構

連線資訊以 JSON 陣列儲存在 `localStorage` 中：

```
// localStorage 中的格式
[
    {
        "id": "conn_1710000000000",
        "name": " 本地開發 SQL",
        "type": 0,                    // 0: SQL Server, 1: PostgreSQL
        "connectionString": "Server=localhost;Database=MyDb;...",
        "readOnly": false
    },
    ...
]
```

9.2.2 唯讀連線

連線設定支援「唯讀」標記：

```
// 唯讀連線不會出現在目的地下拉選單中
if (conn.id !== currentSource && !conn.readOnly) {
    targetSelect.append(`<option value="${conn.id}">${conn.name}</option>`);
}
```

這是一個安全機制——標記為唯讀的連線只能作為來源，永遠不會被誤用為同步目標。

9.2.3 匯入匯出

連線設定支援匯出為 JSON 檔案和匯入：

```
function exportConnections() {
    const data = JSON.stringify(connections, null, 2);
    const blob = new Blob([data], { type: 'application/json' });
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url;
    a.download = `db-connections-${timestamp}.json`;
    a.click();
}
```

匯入時使用智慧合併邏輯——以 (type, name) 為鍵：- 新名稱 □ 新增 - 同名稱 □ 更新連線字串 - 無效資料 □ 略過

```
const keyOf = (c) => `${c.type}|${c.name.trim()}`;
const map = new Map(connections.map(c => [keyOf(c), c]));

imported.forEach(c => {
    const key = keyOf(normalized);
    if (map.has(key)) {
        // 更新現有連線
        existing.connectionString = normalized.connectionString;
        updated++;
    } else {
        // 新增連線
        connections.push(normalized);
        added++;
    }
});
```

9.3 分析與比較流程

使用者點擊「分析與比較」按鈕後的完整流程：

1. 驗證來源和目標已選擇
2. 驗證來源和目標類型相同
3. 顯示載入中動畫
4. 發送 `POST /api/db/compare`
5. 接收 `SyncStatus[]` 結果
6. 呼叫 `renderMatrixUI()` 渲染表格

```
$('#compareBtn').click(async function() {
    const sourceId = $('#sourceConnSelect').val();
    const targetId = $('#targetConnSelect').val();

    if (!sourceId || !targetId) {
        alert('請選擇來源與目的連線。');
        return;
    }

    const source = connections.find(c => c.id === sourceId);
    const target = connections.find(c => c.id === targetId);

    if (source.type !== target.type) {
        alert('來源與目的資料庫類型必須相同。');
        return;
    }
```

```
    // 顯示載入動畫
    $('#syncTable tbody').empty().append(
        '<tr><td colspan="6" class="p-5 text-center">' +
        '<div class="spinner-border text-primary"></div> ' +
        '正在分析來源與目的資料庫...</td></tr>');

    const response = await fetch('/api/db/compare', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ source, target })
    });

    syncItems = await response.json();
    renderMatrixUI();
});
```

9.4 樹狀結構的結果表格

比較結果以**三層樹狀結構**呈現：

```
Schema (dbo)                          ← Schema 層
  Tables (3)                         ← 類型層
      Users          [不存在]       ← 物件層
        IX_Users_Email [ ]         ← 索引層 (子項)
      Orders         [已存在]
      Products       [不存在]
  Views (1)
      vActiveUsers   [不存在]
  Procedures (2)
       GetUser        [不存在]
       UpdateUser     [不存在]
```

9.4.1 分組邏輯

```
function renderMatrixUI() {
    const schemaGroups = {};

    syncItems.forEach((item, index) => {
        // 套用篩選條件
        if (filterType !== "" && item.sourceObject.type.toString() !== filterType) return;
```

```
        if (filterStatus === "exists" && !item.existsInDestination) return;
        if (filterStatus === "missing" && item.existsInDestination) return;
        if (filterSync !== "" && item.status !== filterSync) return;

        const schema = item.sourceObject.schema;
        if (!schemaGroups[schema]) schemaGroups[schema] = {};

        const type = item.sourceObject.type;
        if (!schemaGroups[schema][type]) schemaGroups[schema][type] = [];

        schemaGroups[schema][type].push({ ...item, originalIndex: index });
    });
    ...
}
```

9.4.2 展開/收合

使用 Bootstrap 5 的 Collapse 元件實現樹狀展開/收合：

```
<!-- Schema 列：點擊可收合所有子項 -->
<tr data-bs-toggle="collapse" data-bs-target=".schema-dbo">
    <td><i class="bi bi-chevron-down"></i></td>
    <td colspan="4">dbo</td>
</tr>

<!-- 子項：透過 CSS class 控制顯示 -->
<tr class="collapse show schema-dbo">...</tr>
```

9.4.3 勾選機制

三層勾選盒（全選、Schema 全選、類型全選、個別物件）彼此連動：

```
// Schema 勾選 → 全選該 Schema 下所有物件
$('.schema-check').change(function() {
    const schema = $(this).data('schema');
    const checked = $(this).is(':checked');
    $(`.sync-check[data-schema="${schema}"]:not(:disabled)`).prop('checked', checked);
});

// 類型勾選 → 全選該類型下所有物件
$('.type-check').change(function() {
    const schema = $(this).data('schema');
```

```
    const type = $(this).data('type');
    const checked = $(this).is(':checked');
    $(`.sync-check[data-schema="${schema}"][data-type="${type}"]:not(:disabled)`)
        .prop('checked', checked);
});
```

disabled 的物件不會被勾選——已存在於目標的物件（existsInDestination = true）的勾選盒是 disabled 的，無法被選中進行同步。

9.5 批次同步與進度追蹤

9.5.1 同步流程

```
$('#syncSelectedBtn').click(async function() {
    if (!await showConfirm('同步確認', `確定要同步 ${selected.length} 個物件？`, '開始同步'))
        return;

    isSyncing = true;
    stopRequested = false;

    // 顯示進度條
    $('#syncProgressWrapper').removeClass('d-none');
    updateSyncProgress(0, selected.length);

    for (let i = 0; i < selected.length; i++) {
        if (stopRequested) break;  // 支援中途中斷

        const item = syncItems[idx];
        updateSyncProgress(i + 1, selected.length);

        // 更新該列的狀態為「同步中」
        $(`#sync-row-${idx} .status-cell`).html('  同步中...');

        try {
            const response = await fetch('/api/db/copy', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({
                    source, target,
                    object: item.sourceObject,
                    phase: 0,
                    batchSize: parseInt($('#batchSizeInput').val())
```

```
                })
            });

            if (response.ok) {
                item.status = 'Success';
                // 更新 UI 顯示成功
            } else {
                item.status = 'Error';
                item.message = await response.text();
                // 更新 UI 顯示錯誤
            }
        } catch (err) {
            item.status = 'Error';
            item.message = err.message;
        }
    }

    isSyncing = false;
    // 顯示結果摘要
    showSyncResultBanner(...);
});
```

9.5.2 中斷機制

同步過程中顯示「中斷」按鈕，使用者可以隨時停止：

```
$('#stopSyncBtn').click(function() {
    stopRequested = true;
});

// 迴圈中檢查
for (let i = 0; i < selected.length; i++) {
    if (stopRequested) break;
    ...
}
```

9.5.3 進度條

```
function updateSyncProgress(completed, total) {
    var pct = total > 0 ? Math.round(completed / total * 100) : 0;
    $('#syncProgressBar').css('width', pct + '%').attr('aria-valuenow', pct);
```

```
    $('#syncProgressText').text(pct + '%');
}
```

使用 Bootstrap 5 的 Progress Bar 元件，搭配 `progress-bar-striped progress-bar-animated` 類別產生動態條紋效果。

9.6 匯入匯出連線設定

9.6.1 自訂確認對話框

DbCopy 不使用瀏覽器內建的 `confirm()` 對話框，而是使用自訂的 Bootstrap Modal：

```
function showConfirm(title, message, confirmText, confirmClass) {
    return new Promise(function(resolve) {
        var resolved = false;
        function done(result) {
            if (!resolved) { resolved = true; resolve(result); }
        }

        $('#confirmModalTitle').text(title);
        $('#confirmModalMessage').text(message);
        $('#confirmModalConfirmBtn')
            .attr('class', 'btn btn-sm rounded-pill px-3 ' + confirmClass)
            .text(confirmText);

        var modal = bootstrap.Modal.getOrCreateInstance(modalEl, {
            backdrop: 'static', keyboard: false
        });

        $('#confirmModalConfirmBtn').on('click.confirm', () => { modal.hide(); done(true); });
        $('#confirmModalCancelBtn').on('click.confirm', () => { modal.hide(); done(false); });
        $(modalEl).on('hidden.bs.modal.confirm', () => { done(false); });

        modal.show();
    });
}
```

設計要點：- 回傳 `Promise<boolean>`，可以用 `await` 語法等待使用者選擇 - `backdrop: 'static'` 防止點擊背景關閉 - 命名空間事件（`.confirm`）避免事件衝突 - `resolved` 旗標確保 Promise 只被 resolve 一次

9.6.2 篩選系統

三個下拉式篩選器可以組合使用：

```
<select id="filterObjectType">
    <option value="">所有類型</option>
    <option value="0">Type</option>
    <option value="1">Table Type</option>
    <option value="2">Table</option>
    ...
</select>

<select id="filterTargetStatus">
    <option value="">所有目的狀態</option>
    <option value="missing">不存在 (Missing)</option>
    <option value="exists">已存在 (Exists)</option>
</select>

<select id="filterSyncStatus">
    <option value="">所有同步狀態</option>
    <option value="Pending">準備就緒</option>
    <option value="Success">成功</option>
    <option value="Error">錯誤</option>
</select>
```

篩選邏輯在 `renderMatrixUI()` 中實作，每次篩選條件變更時重新渲染表格。

9.6.3 統計列

頁面底部顯示即時統計：

```
<span>共 <strong id="statTotal">0</strong> 個物件</span>
<span>篩選 <strong id="statFiltered">0</strong> 個</span>
<span>已選 <strong id="statSelected">0</strong> 個</span>
```

下一章預告：UI 的結構已經完成，但一個好的工具需要好的視覺設計。第 10 章將介紹 DbCopy 的 CSS 主題設計——從 CSS 變數系統到漸層配色方案。

10 CSS 主題設計

DbCopy 的視覺設計雖然只有一個 362 行的 CSS 檔案，卻涵蓋了完整的色彩系統、漸層配色、物件類型色碼、響應式設計。

10.1 CSS 變數系統

所有顏色集中定義在 `:root` 偽類中，方便全域管理：

```
:root {
    /* 主色調 */
    --db-primary: #2563eb;          /* 藍色 */
    --db-primary-hover: #1d4ed8;    /* 深藍色 */
    --db-primary-soft: #dbeafe;     /* 淺藍色 */

    /* 語意色彩 */
    --db-success: #059669;          /* 綠色 */
    --db-warning: #ea580c;          /* 橘色 */
    --db-danger: #dc2626;           /* 紅色 */
    --db-info: #0891b2;             /* 青色 */

    /* 強調色 */
    --db-indigo: #6366f1;           /* 靛藍色 */
    --db-purple: #7c3aed;           /* 紫色 */

    /* 灰階 */
    --db-gray-50: #f9fafb;
    --db-gray-100: #f3f4f6;
    --db-gray-200: #e5e7eb;
    --db-gray-300: #d1d5db;
    --db-gray-500: #6b7280;
    --db-gray-600: #4b5563;
    --db-gray-700: #374151;
    --db-gray-800: #1f2937;

    /* 物件類型色彩 */
    --color-table: #2563eb;         /* 藍色 —資料表 */
    --color-view: #059669;          /* 綠色 —檢視 */
```

```
    --color-procedure: #7c3aed;     /* 紫色 —預存程序 */
    --color-function: #db2777;      /* 粉紅色 —函數 */
    --color-type: #0891b2;          /* 青色 —使用者定義型別 */
    --color-table-type: #0369a1;    /* 深青色 —使用者定義資料表型別 */
}
```

10.1.1 色彩系統的設計原則

1. **Tailwind CSS 色盤**：所有顏色值取自 Tailwind CSS 的色盤系統，確保色彩的和諧度和對比度
2. **主色調為藍色**：藍色傳遞「專業」、「信任」的視覺訊號，適合工具類軟體
3. **每個物件類型一個顏色**：使用者可以一眼區分不同類型的物件，不需要閱讀文字
4. **三段式色階**：每個主要色彩提供 Normal、Hover、Soft 三個變體（如 `--db-primary`、`--db-primary-hover`、`--db-primary-soft`）

10.2 漸層配色方案

DbCopy 大量使用 CSS 漸層（Gradient），讓介面更具深度感和現代感：

10.2.1 背景漸層

```
body {
    background: linear-gradient(135deg, #f0f9ff 0%, #f8fafc 50%, #fef3c7 100%);
    min-height: 100vh;
}
```

從左上到右下，由淺藍 □ 灰白 □ 淺黃的三段漸層，營造出溫暖而專業的背景。

10.2.2 按鈕漸層

```
.btn-primary {
    background: linear-gradient(135deg, var(--db-primary) 0%, var(--db-indigo) 100%);
    border: none;
    box-shadow: 0 4px 6px -1px rgba(37, 99, 235, 0.3),
                0 2px 4px -1px rgba(37, 99, 235, 0.2);
    transition: all 0.3s ease;
}

.btn-primary:hover {
```

```
    background: linear-gradient(135deg, var(--db-primary-hover) 0%, var(--db-purple) 100%);
    transform: translateY(-2px);    /* 懸浮時微微上升 */
    box-shadow: 0 6px 8px -1px rgba(37, 99, 235, 0.4),
                0 4px 6px -1px rgba(37, 99, 235, 0.3);
}
```

Hover 效果三重奏：1. 漸層色彩加深 2. 微微上移 2px（`translateY(-2px)`）3. 陰影加強

10.2.3 表頭漸層

```
.table thead th {
    background: linear-gradient(135deg, var(--db-primary) 0%, var(--db-indigo) 100%) !important
    color: #ffffff !important;
    text-shadow: 0 1px 2px rgba(0, 0, 0, 0.2);
}
```

漸層表頭搭配白色文字和文字陰影，確保可讀性的同時增加視覺層次。

10.2.4 捲軸漸層

```
::-webkit-scrollbar-thumb {
    background: linear-gradient(135deg, var(--db-primary) 0%, var(--db-indigo) 100%);
    border-radius: 5px;
}
```

即使是捲軸也保持與整體配色一致。

10.3 物件類型色彩編碼

不同類型的資料庫物件使用不同的色彩標記：

```
.badge-table     { background-color: var(--color-table); }      /* 藍色 */
.badge-view      { background-color: var(--color-view); }       /* 綠色 */
.badge-procedure { background-color: var(--color-procedure); }  /* 紫色 */
.badge-function  { background-color: var(--color-function); }   /* 粉紅色 */
.badge-type      { background-color: var(--color-type); }       /* 青色 */
.badge-table-type { background-color: var(--color-table-type); } /* 深青色 */
```

10.3.1 樹狀表格的列樣式

```
/* Schema 列 —淺藍色背景 + 左邊框 */
.schema-row {
    background: linear-gradient(90deg, #dbeafe 0%, #e0e7ff 100%) !important;
    border-left: 3px solid var(--db-primary);
    font-weight: 600;
}

/* 類型列 —更淺的背景 */
.type-row {
    background: linear-gradient(90deg, #f0f9ff 0%, #fef3c7 100%) !important;
    border-left: 3px solid var(--db-indigo);
}

/* 物件列 —白色背景 */
.object-row {
    border-left: 4px solid transparent;
    background-color: #ffffff;
}

/* 物件列 Hover —淺藍到淺黃的漸層 */
.object-row:hover {
    background: linear-gradient(90deg, #dbeafe 0%, #fef9c3 100%) !important;
    border-left-color: var(--db-warning);
    transition: all 0.2s ease-in-out;
}

/* 索引列 —淺黃色背景 + 斜體 */
.index-row {
    background-color: #fefce8 !important;
    font-style: italic;
    border-left: 3px solid var(--db-warning-soft);
}
```

每種列類型使用不同的左邊框顏色和背景漸層，讓使用者一眼就能區分層級。

10.3.2 狀態 Badge

```
/* 「已存在」—橘色外框 */
.badge-outline-warning {
    color: #92400e;
```

```
    border: 1px solid #f59e0b;
    background: linear-gradient(135deg, #fffbeb 0%, #fef3c7 100%);
    font-weight: 600;
}

/* 「不存在」—綠色外框 */
.badge-outline-success {
    color: #065f46;
    border: 1px solid #10b981;
    background: linear-gradient(135deg, #ecfdf5 0%, #d1fae5 100%);
    font-weight: 600;
}

/* 「準備就緒」—灰色外框 */
.badge-outline-secondary {
    color: var(--db-gray-700);
    border: 1px solid var(--db-gray-300);
    background: linear-gradient(135deg, #f9fafb 0%, #e5e7eb 100%);
    font-weight: 600;
}
```

10.4 響應式設計考量

10.4.1 字型大小適應

```
html {
    font-size: 14px;
}

@media (min-width: 768px) {
    html {
        font-size: 16px;
    }
}
```

小螢幕使用 14px 基礎字型大小，平板以上使用 16px。使用 `rem` 單位的元素會自動按比例縮放。

10.4.2 表格滾動區域

```
<div class="table-responsive" style="max-height: calc(100vh - 310px); overflow-y: auto;">
```

表格區域使用 `calc(100vh - 310px)` 計算最大高度，確保在任何螢幕大小下都不會超出可視範圍。`310px` 是工具列 + 篩選列 + 統計列的約略高度。

10.4.3 字型選擇

```
body {
    font-family: 'Inter', -apple-system, BlinkMacSystemFont, "Segoe UI",
                 Roboto, Helvetica, Arial, sans-serif;
}
```

Inter 字型是專為 UI 設計的開源字型，數字字距等寬（tabular numbers），非常適合顯示資料表格。透過 Google Fonts CDN 載入，並提供系統字型作為 fallback。

10.4.4 Modal 對話框

```
.modal-header {
    background: linear-gradient(135deg, var(--db-primary) 0%, var(--db-indigo) 100%);
    color: #ffffff;
    border-bottom: 2px solid var(--db-primary-hover);
}

.modal-header .btn-close {
    filter: brightness(0) invert(1);    /* 白色關閉按鈕 */
}

.modal-header.bg-danger {
    background: linear-gradient(135deg, var(--db-danger) 0%, #ef4444 100%) !important;
}
```

一般 Modal 使用藍色漸層表頭，錯誤 Modal 使用紅色漸層表頭。`filter: brightness(0) invert(1)` 是一個巧妙的技巧——將深色關閉按鈕反轉為白色，不需要替換圖示。

下一章預告：UI 層的設計告一段落。第 11 章將深入 `Program.cs`——應用程式的進入點，包括 Serilog 配置、嵌入式靜態檔案、連接埠衝突自動偵測等基礎設施。

11 Program.cs — 應用程式進入點

`Program.cs`（182 行）是 DbCopy 的起點。它負責設定日誌、註冊服務、配置中介軟體、處理連接埠衝突、啟動 Web 伺服器，並在啟動後自動開啟瀏覽器。

11.1 Serilog 日誌配置

```
Log.Logger = new LoggerConfiguration()
    .WriteTo.Console()
    .WriteTo.File("Logs/log-.txt", rollingInterval: RollingInterval.Day)
    .CreateLogger();

builder.Host.UseSerilog();
```

11.1.1 為什麼選擇 Serilog？

ASP.NET Core 內建的 `Microsoft.Extensions.Logging` 提供了日誌抽象層，但它的輸出格式和目的地（Sink）選項有限。Serilog 提供了：

功能	內建 Logging	Serilog
結構化日誌	基本支援	完整支援
Console 輸出	有	有（更美觀）
檔案輸出	無	有（含滾動）
滾動策略	無	Day/Hour/Size
訊息模板	基本	豐富

11.1.2 日誌輸出配置

- **Console**：開發時在終端機中即時查看日誌
- **File**：`Logs/log-20260317.txt` 格式的每日滾動檔案，方便事後排查問題

`RollingInterval.Day` 表示每天建立一個新的日誌檔案，檔名包含日期。

11.2 依賴注入容器設定

```
builder.Services.AddRazorPages();           // 啟用 Razor Pages
builder.Services.AddControllers();           // 啟用 API Controllers
builder.Services.AddScoped<SqlServerService>();    // 每個 HTTP 請求一個實例
builder.Services.AddScoped<PostgreSqlService>();   // 每個 HTTP 請求一個實例
```

11.2.1 生命週期選擇：Scoped

生命週期	說明	適用場景
Singleton	整個應用程式生命週期只有一個實例	無狀態的工具類
Scoped	**每個 HTTP 請求一個實例**	**含有 Logger 的服務**
Transient	每次注入都建立新實例	輕量級無狀態服務

`SqlServerService` 和 `PostgreSqlService` 使用 `Scoped` 是因為它們注入了 `ILogger<T>`。Logger 本身是 Scoped 的，所以使用它的服務也應該是 Scoped 的。

11.2.2 中介軟體管線

```
var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
    app.UseHttpsRedirection();     // 僅在非開發環境啟用 HTTPS 重導向
}

app.UseRouting();
// 注意：沒有 UseAuthorization() —因為沒有配置認證/授權方案

// 嵌入式靜態檔案
var embeddedProvider = new ManifestEmbeddedFileProvider(
    typeof(Program).Assembly, "wwwroot");
app.UseStaticFiles(new StaticFileOptions
{
    FileProvider = embeddedProvider
});
```

```
app.MapRazorPages();      // 映射 Razor Pages 路由
app.MapControllers();     // 映射 API Controllers 路由
```

設計決策：

1. **不使用 `UseAuthorization()`**：DbCopy 不需要認證/授權，因為它是本地工具。加上這個中介軟體只會造成不必要的警告。
2. **HTTPS 重導向僅在生產環境**：開發時使用 HTTP 更方便，避免憑證問題。
3. **路由順序**：先靜態檔案、再 Razor Pages、最後 API Controllers。

11.3 嵌入式靜態檔案

這是 DbCopy 實現「單一執行檔」的核心技術。

11.3.1 問題

傳統的 ASP.NET Core 應用在部署時需要一個 `wwwroot` 資料夾，裡面放著 CSS、JavaScript、圖片等靜態檔案。但 DbCopy 的目標是一個單一的 `.exe` 檔案，不需要額外的資料夾。

11.3.2 解決方案

步驟一：在 `.csproj` 中將 `wwwroot` 所有檔案標記為嵌入資源：

```
<PropertyGroup>
    <GenerateEmbeddedFilesManifest>true</GenerateEmbeddedFilesManifest>
</PropertyGroup>

<ItemGroup>
    <EmbeddedResource Include="wwwroot\**" />
</ItemGroup>
```

步驟二：在 `Program.cs` 中使用 `ManifestEmbeddedFileProvider`：

```
var embeddedProvider = new ManifestEmbeddedFileProvider(
    typeof(Program).Assembly, "wwwroot");

app.UseStaticFiles(new StaticFileOptions
{
    FileProvider = embeddedProvider
});
```

11.3.3 運作原理

1. **建置時**：MSBuild 將 `wwwroot/**` 的所有檔案嵌入到組件（.dll）中
2. **執行時**：`ManifestEmbeddedFileProvider` 從組件中讀取嵌入的檔案
3. **瀏覽器請求**：`/css/site.css` □ 中介軟體從組件中找到嵌入的 `wwwroot/css/site.css` □ 回傳給瀏覽器

這樣一來，即使只有一個 `.exe` 檔案，所有靜態資源都可以正常存取。

11.3.4 需要的 NuGet 套件

```
<PackageReference Include="Microsoft.Extensions.FileProviders.Embedded" Version="10.0.0" />
```

11.4 連接埠衝突自動偵測

當使用者同時運行多個 DbCopy 實例，或其他服務已佔用預設連接埠時，程式會自動尋找可用的連接埠：

```
var configuredUrls = builder.Configuration["urls"];
if (!string.IsNullOrWhiteSpace(configuredUrls))
{
    var adjustedUrls = AdjustUrlsForPortConflicts(configuredUrls);
    if (!string.Equals(adjustedUrls, configuredUrls, StringComparison.Ordinal))
    {
        builder.WebHost.UseUrls(adjustedUrls);
        Log.Warning("Detected occupied port(s). Switched URLs from {OriginalUrls} to {AdjustedU
            configuredUrls, adjustedUrls);
    }
}
```

11.4.1 核心演算法

```
private static string AdjustUrlsForPortConflicts(string urls)
{
    var candidates = urls.Split(';', StringSplitOptions.RemoveEmptyEntries | ...);
    var reservedPorts = new HashSet<int>();  // 已預留的連接埠
    var adjusted = new string[candidates.Length];

    for (var i = 0; i < candidates.Length; i++)
```

```
    {
        if (!Uri.TryCreate(current, UriKind.Absolute, out var uri)) { ... continue; }
        if (!IsLocalHost(uri.Host)) { ... continue; }

        var port = uri.Port;
        if (reservedPorts.Contains(port) || !IsPortAvailable(port))
        {
            var newPort = FindNextAvailablePort(port + 1, reservedPorts);
            var builder = new UriBuilder(uri) { Port = newPort };
            adjusted[i] = builder.Uri.ToString().TrimEnd('/');
            reservedPorts.Add(newPort);
        }
        else
        {
            adjusted[i] = current;
            reservedPorts.Add(port);
        }
    }

    return string.Join(';', adjusted);
}
```

11.4.2 連接埠可用性檢查

```
private static bool IsPortAvailable(int port)
{
    return TryBind(IPAddress.Loopback, port)         // IPv4: 127.0.0.1
        && TryBind(IPAddress.IPv6Loopback, port);    // IPv6: ::1
}

private static bool TryBind(IPAddress ipAddress, int port)
{
    try
    {
        using var listener = new TcpListener(ipAddress, port);
        listener.Start();
        listener.Stop();
        return true;
    }
    catch (SocketException) { return false; }
}
```

透過實際嘗試綁定 TCP 連接埠來檢查可用性。需要同時檢查 IPv4 和 IPv6，因為某些系統只佔用了其

中一個。

11.4.3 reservedPorts 的作用

當配置了多個 URL（如 `http://localhost:5281;https://localhost:7256`）時，`reservedPorts` 集合確保自動選擇的新連接埠不會與已調整的連接埠衝突。

11.5 自動開啟瀏覽器

```
app.Lifetime.ApplicationStarted.Register(() =>
{
    var url = app.Urls.FirstOrDefault() ?? "http://localhost:5000";
    Log.Information("Opening browser at {Url}", url);
    System.Diagnostics.Process.Start(new System.Diagnostics.ProcessStartInfo
    {
        FileName = url,
        UseShellExecute = true
    });
});
```

11.5.1 設計要點

1. `ApplicationStarted` **事件**：確保在 Web 伺服器完全啟動後才開啟瀏覽器，避免瀏覽器開啟時伺服器尚未就緒
2. `UseShellExecute = true`：告訴作業系統用預設瀏覽器開啟 URL。這在 Windows、macOS、Linux 上都能正確運作
3. `app.Urls.FirstOrDefault()`：使用實際綁定的 URL（可能已經被連接埠衝突調整過），而非配置的 URL

11.5.2 跨平台相容性

`Process.Start` 搭配 `UseShellExecute = true`：- **Windows**：等同於 `start http://localhost:5281` - **macOS**：等同於 `open http://localhost:5281` - **Linux**：等同於 `xdg-open http://localhost:5281`

.NET 的 `UseShellExecute` 會自動使用對應平台的機制。

11.5.3 全域錯誤處理

```
try
{
    Log.Information("Starting web host");
    ...
    app.Run();
}
catch (Exception ex)
{
    Log.Fatal(ex, "Host terminated unexpectedly");
}
finally
{
    Log.CloseAndFlush();    // 確保所有日誌都寫入磁碟
}
```

`Log.CloseAndFlush()` 非常重要——Serilog 使用非同步緩衝寫入檔案，如果不在應用程式結束前刷新，最後幾行日誌可能會遺失。

下一章預告：程式碼已經完成，接下來要把它變成可以發佈的產品。第 12 章將介紹跨平台建置與發佈——如何用一個腳本為五個平台產出單一執行檔。

12 跨平台建置與發佈

12.1 專案檔設定解析

DbCopy.csproj 是整個專案的配置核心：

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <GenerateEmbeddedFilesManifest>true</GenerateEmbeddedFilesManifest>
    <ApplicationIcon>app.ico</ApplicationIcon>
    <Deterministic>false</Deterministic>
    <AssemblyVersion>1.0.0.*</AssemblyVersion>
    <FileVersion>1.0.0.*</FileVersion>
    <NoWarn>$(NoWarn);CS7035</NoWarn>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Dapper" Version="2.1.66" />
    <PackageReference Include="Microsoft.Data.SqlClient" Version="6.1.4" />
    <PackageReference Include="Microsoft.Extensions.FileProviders.Embedded" Version="10.0.0" />
    <PackageReference Include="Npgsql" Version="10.0.1" />
    <PackageReference Include="Serilog.AspNetCore" Version="10.0.0" />
    <PackageReference Include="Serilog.Sinks.File" Version="7.0.0" />
  </ItemGroup>

  <ItemGroup>
    <EmbeddedResource Include="wwwroot\**" />
  </ItemGroup>
</Project>
```

12.1.1 各設定項解析

設定	說明
`TargetFramework=net10.0`	目標框架為.NET 10.0
`Nullable=enable`	啟用 C# 可空參考型別分析
`ImplicitUsings=enable`	自動匯入常用命名空間
`GenerateEmbeddedFilesManifest=true`	為嵌入資源產生清單檔
`ApplicationIcon=app.ico`	Windows 執行檔圖示
`Deterministic=false`	允許 * 萬用字元版本號
`AssemblyVersion=1.0.0.*`	建置版本自動遞增
`NoWarn=CS7035`	抑制萬用字元版本號的警告

12.1.2 版本號的萬用字元

```
AssemblyVersion=1.0.0.*
```

* 會在每次建置時自動替換為基於日期和時間的數值：- 第三段（Build）：自 2000-01-01 以來的天數 - 第四段（Revision）：當天午夜以來的秒數 / 2

這讓每次建置都有唯一的版本號，方便追蹤問題。

12.1.3 套件依賴

套件	版本	用途
Dapper	2.1.66	微型 ORM
Microsoft.Data.SqlClient	6.1.4	SQL Server 連線驅動
Microsoft.Extensions.FileProviders.Embedded	10.0.0	嵌入式靜態檔案
Npgsql	10.0.1	PostgreSQL 連線驅動
Serilog.AspNetCore	10.0.0	結構化日誌
Serilog.Sinks.File	7.0.0	檔案日誌輸出

依賴最小化原則：只使用必要的套件，不引入不需要的框架（如 Entity Framework、AutoMapper 等）。總共只有 6 個直接依賴。

12.2 五平台單一執行檔發佈

`publish.sh` 是一個簡潔的 Bash 腳本（37 行），為五個目標平台產出獨立的單一執行檔：

```bash
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT="$ROOT_DIR/DbCopy.csproj"
PUBLISH_DIR="$ROOT_DIR/publish"

RIDS=(
    "win-x64"
    "linux-x64"
    "linux-arm64"
    "osx-x64"
    "osx-arm64"
)

rm -rf "$PUBLISH_DIR"
mkdir -p "$PUBLISH_DIR"

for rid in "${RIDS[@]}"; do
    echo "Publishing for $rid..."
    dotnet publish "$PROJECT" \
        -c Release \
        -r "$rid" \
        --self-contained true \
        -p:PublishSingleFile=true \
        -p:EnableCompressionInSingleFile=true \
        -o "$PUBLISH_DIR/$rid"
done
```

12.2.1 發佈參數解析

參數	說明
`-c Release`	使用 Release 配置（啟用最佳化）
`-r win-x64`	目標執行環境（Runtime Identifier）
`--self-contained true`	包含.NET Runtime（不需要預先安裝）
`-p:PublishSingleFile=true`	打包成單一執行檔
`-p:EnableCompressionInSingleFile=true`	壓縮單一檔案（減少約 30% 大小）

12.2.2 五個目標平台

RID	作業系統	架構	常見裝置
win-x64	Windows	Intel/AMD 64bit	大多數 Windows PC
linux-x64	Linux	Intel/AMD 64bit	伺服器、WSL
linux-arm64	Linux	ARM 64bit	樹莓派 4、AWS Graviton
osx-x64	macOS	Intel	2020 前的 Mac
osx-arm64	macOS	Apple Silicon	M1/M2/M3/M4 Mac

12.2.3 產出結構

```
publish/
  win-x64/
     DbCopy.exe          (~60 MB)
  linux-x64/
     DbCopy              (~60 MB)
  linux-arm64/
     DbCopy              (~55 MB)
  osx-x64/
     DbCopy              (~60 MB)
  osx-arm64/
      DbCopy              (~55 MB)
```

每個目標資料夾只有一個檔案。使用者下載後直接執行即可，不需要安裝.NET Runtime。

12.3 語意化版本控制

`release.sh`（67 行）是一個互動式腳本，實作了語意化版本控制（Semantic Versioning）：

```
#!/usr/bin/env bash
set -euo pipefail

# 1. 確保工作目錄乾淨
if ! git diff --quiet --ignore-submodules HEAD --; then
    echo "Working tree has uncommitted changes." >&2
    exit 1
fi

# 2. 取得最新的版本標籤（比較本地和遠端）
latest_local_tag="$(git tag --list 'v[0-9]*.[0-9]*.[0-9]*' --sort=-v:refname | head -n 1)"
latest_remote_tag="$(git ls-remote --tags --refs origin 'v[0-9]*.[0-9]*.[0-9]*' ...)"

# 3. 取較大的版本號
```

```
latest_tag="$(printf '%s\n%s\n' "$latest_local_tag" "$latest_remote_tag" | sort -V | tail -n 1)

# 4. 建議下一個版本號 (Patch +1)
if [[ "$latest_tag" =~ ^v([0-9]+)\.([0-9]+)\.([0-9]+)$ ]]; then
    suggested_tag="v${major}.${minor}.$((patch + 1))"
fi

# 5. 互動式確認
read -r -p "Release tag [${suggested_tag}]: " input_tag
tag="${input_tag:-$suggested_tag}"

# 6. 驗證標籤格式和唯一性
if git rev-parse -q --verify "refs/tags/$tag" >/dev/null; then
    echo "Tag already exists locally: $tag" >&2
    exit 1
fi

# 7. 建立並推送標籤
git tag "$tag"
git push origin "$tag"
```

12.3.1 安全機制

1. **工作目錄檢查**：未提交的變更和未追蹤的檔案都會阻止發佈
2. **本地 + 遠端比較**：取兩者中較大的版本號，防止多人同時發佈時版本衝突
3. **格式驗證**：確保標籤符合 v<major>.<minor>.<patch> 格式
4. **重複檢查**：確保標籤在本地和遠端都不存在

12.4 嵌入式資源的運作原理

12.4.1 建置時流程

```
原始碼                          建置輸出

wwwroot/                       DbCopy.dll
   css/site.css     →           嵌入資源: DbCopy.wwwroot.css.site.css
   js/site.js       →           嵌入資源: DbCopy.wwwroot.js.site.js
   lib/             →           嵌入資源: DbCopy.wwwroot.lib.*
   favicon.ico      →           嵌入資源: DbCopy.wwwroot.favicon.ico
```

GenerateEmbeddedFilesManifest=true 會額外產生一個清單（manifest），記錄所有嵌入資源的原始路徑和組件中的名稱對應。

12.4.2 執行時流程

```
瀏覽器請求: GET /css/site.css
    ↓
ASP.NET Core Static Files 中介軟體
    ↓
ManifestEmbeddedFileProvider
    ↓
查詢清單: /css/site.css → DbCopy.wwwroot.css.site.css
    ↓
從 Assembly 載入嵌入資源
    ↓
回傳 HTTP Response (Content-Type: text/css)
```

12.4.3 與 Single File Publishing 的配合

當使用 `PublishSingleFile=true` 時：

1. 所有.dll（包含嵌入資源的 DbCopy.dll）被打包進一個.exe
2. 執行時.NET Runtime 從.exe 中解壓.dll
3. `ManifestEmbeddedFileProvider` 從.dll 中讀取嵌入資源

整個鏈條確保了「一個檔案即可運行」的目標。

下一章預告：程式碼和部署都完成了。第 13 章將回顧安全性設計——DbCopy 如何防範 SQL 注入、處理識別符跳脫、保護連線字串。

13 安全性設計

資料庫工具本身就是高權限的軟體——它可以讀取結構、複製資料、建立物件。因此安全性設計不是事後補強，而是從第一行程式碼就需要考慮的事。

13.1 SQL 注入防護

SQL 注入是資料庫應用最常見的安全漏洞。DbCopy 的防護策略分為兩個層次。

13.1.1 第一層：參數化查詢（Dapper）

所有使用者可控的值（Schema 名稱、物件名稱）都透過 Dapper 的參數化查詢傳入：

```
// 安全：@Schema 和 @Name 是參數化的
const string sql = "SELECT 1 FROM sys.tables WHERE SCHEMA_NAME(schema_id) = @Schema AND name =
await conn.ExecuteScalarWithLogAsync<int?>(logger, sql, new { obj.Schema, obj.Name });
```

Dapper 會將 `@Schema` 和 `@Name` 轉換為 SQL 參數（`SqlParameter` / `NpgsqlParameter`），確保值永遠不會被當作 SQL 程式碼執行。

13.1.2 哪些場景無法使用參數化？

SQL 的語法限制使得某些位置**不能使用參數**：

```
--  不合法：表名不能用參數
SELECT * FROM @TableName

--  不合法：Schema 名不能用參數
CREATE SCHEMA @SchemaName

--  不合法：DDL 語句不能用參數
CREATE TABLE @TableName (...)
```

這些場景只能使用**識別符跳脫**來防護。

13.2 識別符跳脫策略

13.2.1 SQL Server：方括號跳脫

SQL Server 使用方括號 `[...]` 引用識別符。方括號內的 `]` 需要雙重為 `]]`：

```
// SQL Server 的識別符跳脫
$"SELECT * FROM [{obj.Schema.Replace("]", "]]")}].[{obj.Name.Replace("]", "]]")}]"

// 範例：
// Schema = "dbo"     → [dbo]
// Name = "Users"     → [Users]
// Name = "User]s"    → [User]]s]     ← 內部的 ] 被跳脫
```

這個跳脫在整個 `SqlServerService` 中被一致地使用：

```
// CopyTableDataAsync 中的 SELECT
var sql = $"SELECT * FROM [{obj.Schema.Replace("]", "]]")}].[{obj.Name.Replace("]", "]]")}]";

// GetRowCountAsync 中的 COUNT
$"SELECT COUNT_BIG(*) FROM [{obj.Schema.Replace("]", "]]")}].[{obj.Name.Replace("]", "]]")}]"

// SqlBulkCopy 的目的表名
bulkCopy.DestinationTableName = $"[{obj.Schema.Replace("]", "]]")}].[{obj.Name.Replace("]", "]]

// EnsureSchemaExistsAsync
$"CREATE SCHEMA [{schema.Replace("]", "]]")}]"
```

13.2.2 PostgreSQL：雙引號跳脫

PostgreSQL 使用雙引號 `"..."` 引用識別符。雙引號內的 `"` 需要雙重為 `""`：

```
// PostgreSQL 的識別符跳脫
$"CREATE SCHEMA \"{schema.Replace("\"", "\"\"")}\""

// 但在多數地方，PostgreSQL 直接使用雙引號而不做額外跳脫
// 因為 Schema/Name 來自 pg_catalog，本身就是合法的識別符
$"SELECT COUNT(*)::bigint FROM \"{obj.Schema}\".\"{obj.Name}\""
```

13.2.3 PostgreSQL 特有：PL/pgSQL 中的 format('%I', ...)

在 PL/pgSQL 匿名區塊中，使用 `format('%I', ...)` 函數來安全地引用識別符：

```
var setValSql = $"""
    DO $$
    DECLARE max_val bigint;
    BEGIN
        EXECUTE format('SELECT MAX(%I) FROM %I.%I',
            '{escapedCol}', '{escapedSchema}', '{escapedName}') INTO max_val;
        IF max_val IS NULL THEN
            PERFORM setval(format('%I.%I', '{escapedSeqSchema}', '{escapedSeqName}'), 1, false)
        ELSE
            PERFORM setval(format('%I.%I', '{escapedSeqSchema}', '{escapedSeqName}'), max_val,
        END IF;
    END $$;
    """;
```

這裡有兩層防護：

1. `EscapePgLiteral()`：將單引號 `'` 跳脫為 `''`，防止從字串常值中逃逸
2. `format('%I', ...)`：PostgreSQL 內建函數，自動為識別符加上雙引號並處理跳脫

13.2.4 QUOTENAME — SQL Server 內建的安全識別符

在某些 SQL 查詢中，DbCopy 使用 `QUOTENAME()` 函數：

```
SELECT OBJECT_DEFINITION(OBJECT_ID(QUOTENAME(@Schema) + '.' + QUOTENAME(@Name)))
```

`QUOTENAME()` 是 SQL Server 內建的函數，等同於 `[` + 跳脫 `]` + `]`。它與參數化配合使用——`@Schema` 和 `@Name` 是參數化的值，`QUOTENAME()` 在 SQL 伺服器端處理識別符引用。

13.3 連線字串的安全考量

13.3.1 傳輸安全

連線字串包含帳號密碼，屬於高度敏感資訊。DbCopy 的設計：

面向	做法
存儲	存在瀏覽器的 `localStorage` 中（僅存在本地）
傳輸	透過 POST Request Body 傳送（不在 URL 中）

面向	做法
日誌	不記錄連線字串（僅記錄 SQL 語句）
顯示	在連線管理 Modal 中可見，但不在主頁面顯示

13.3.2 未實作認證機制

DbCopy **刻意不實作**認證/授權機制：

```
// Program.cs
// Fix #13: UseAuthorization removed — no auth scheme or policies are configured
```

原因：DbCopy 設計為**本地工具**，預設綁定到 `localhost`。如果需要在網路上暴露，應該透過反向代理（Nginx/Caddy）或 VPN 來保護，而不是在工具本身加入簡陋的認證機制。

13.3.3 唯讀連線的保護

前端提供了「唯讀」標記功能：

```
// 唯讀連線不會出現在目標選擇下拉中
if (conn.id !== currentSource && !conn.readOnly) {
    targetSelect.append(`<option>${conn.name}</option>`);
}
```

這是一個**前端層級的保護**，防止使用者誤將生產資料庫設為同步目標。但請注意，這不是一個安全機制——惡意使用者可以透過直接呼叫 API 繞過此限制。它的目的是**防誤操作**，而非防攻擊。

13.3.4 DDL 語句的安全性

`GetObjectDefinitionAsync` 產生的 DDL 語句會直接在目標資料庫中執行：

```
var definition = await GetObjectDefinitionAsync(sourceConnectionString, obj);
await conn.ExecuteWithLogAsync(logger, definition);
```

這是否構成安全風險？

不構成，因為 DDL 定義來自**來源資料庫的系統目錄**，而非使用者輸入。如果攻擊者能夠控制來源資料庫的系統目錄，那麼他已經擁有了 DBA 權限，不需要透過 DbCopy 來發動攻擊。

13.3.5 安全設計的哲學

DbCopy 的安全設計遵循一個原則：

> **保護層級應與威脅模型匹配。**

- 本地工具 → 不需要網路層認證
- 系統目錄查詢 → 不需要防禦 SQL 注入（系統目錄不是使用者輸入）
- 動態識別符 → 需要識別符跳脫
- 連線字串 → 不記錄到日誌

過度的安全措施會增加複雜度而不增加安全性。

> **下一章預告**：安全性之後，第 14 章將探討效能與可靠性——非同步設計模式、平行化策略、批次處理，以及如何優雅地處理部分失敗。

14 效能與可靠性

DbCopy 需要處理大量的資料庫查詢和資料傳輸。本章討論系統中使用的效能優化策略和可靠性保障機制。

14.1 非同步設計模式

DbCopy 的所有 I/O 操作都使用 `async/await` 模式：

```
// 每個方法都是非同步的
public async Task<bool> TestConnectionAsync(string connectionString)
{
    await using var conn = new SqlConnection(connectionString);
    await conn.OpenAsync();
    return true;
}
```

14.1.1 為什麼非同步很重要？

DbCopy 是一個 Web 應用，每個 HTTP 請求都由一個執行緒處理。如果使用同步 I/O：

```
同步模式：
Thread 1 → 發送 SQL → [等待回應...............] → 處理結果
                        ↑ 執行緒被阻塞，無法處理其他請求

非同步模式：
Thread 1 → 發送 SQL → [釋放執行緒] ← 處理結果
                        ↑ 執行緒可以處理其他請求
```

對於 DbCopy 這種需要大量資料庫往返的工具，非同步模式確保了：1. **Web 伺服器不會因為長時間查詢而無回應** 2. **同時處理多個使用者的請求**（雖然通常只有一個使用者）3. **I/O 等待時間不浪費 CPU 資源**

14.1.2 `await using` 模式

```
await using var conn = new SqlConnection(connectionString);
```

`await using` 確保非同步資源在作用域結束時被正確釋放。這比 `using` 更適合非同步情境，因為資源的釋放本身也可能是非同步操作（如關閉網路連線）。

14.2 Task.WhenAll 平行化策略

DbCopy 在 `Compare` 端點中大量使用 `Task.WhenAll` 來平行化獨立的查詢：

14.2.1 策略一：來源與目標平行查詢

```
// 這兩個查詢查的是不同的資料庫，完全獨立
var existsTask = targetService.CheckObjectExistsAsync(request.Target.ConnectionString, obj);
var depsTask = sourceService.GetDependenciesAsync(request.Source.ConnectionString, obj);
await Task.WhenAll(existsTask, depsTask);

var exists = existsTask.Result;
var deps = depsTask.Result;
```

效果：如果每個查詢需要 50ms，循序執行需要 100ms，平行執行只需要 ~50ms。

14.2.2 策略二：四個資料表查詢平行化

```
var sourceRowsTask = SafeGetRowCount(sourceService, ...);
var sourceIndexesTask = SafeGetIndexes(sourceService, ...);
var targetRowsTask = exists
    ? SafeGetRowCount(targetService, ...)
    : Task.FromResult<long?>(null);
var targetIndexesTask = exists
    ? SafeGetIndexes(targetService, ...)
    : Task.FromResult(new List<DbIndex>());

await Task.WhenAll(sourceRowsTask, sourceIndexesTask, targetRowsTask, targetIndexesTask);
```

效果：四個查詢同時發出，總等待時間等於最慢的那個查詢。

14.2.3 條件性跳過

```
var targetRowsTask = exists
    ? SafeGetRowCount(targetService, ...)
    : Task.FromResult<long?>(null);  // 目標不存在，直接回傳 null，零成本
```

如果目標物件不存在，使用 `Task.FromResult` 回傳預設值。這避免了不必要的資料庫查詢。

14.2.4 注意事項

平行化的前提是查詢之間**真正獨立**。以下情況不能平行化：

```
//   錯誤：先取定義，再執行定義，有相依性
var definition = await GetObjectDefinitionAsync(...);  // 必須先完成
await conn.ExecuteAsync(definition);                   // 依賴上一步的結果
```

14.3 批次處理與串流

14.3.1 SQL Server：SqlBulkCopy 的批次模式

```
using var bulkCopy = new SqlBulkCopy(targetConn, ...);
bulkCopy.BatchSize = batchSize;  // 預設 1000
bulkCopy.BulkCopyTimeout = 600;  // 10 分鐘

await bulkCopy.WriteToServerAsync(reader);
```

`BatchSize` 控制每次提交到伺服器的列數：- **太小**（如 10）：頻繁的網路往返，效率低 - **太大**（如 100000）：伺服器端暫存大量資料，佔用記憶體和交易日誌空間 - **預設 1000**：在效能和資源使用之間取得平衡

14.3.2 PostgreSQL：COPY 協定的串流模式

```
// 二進位模式 —逐列串流，不需要一次載入所有資料
await using var importer = await targetConn.BeginBinaryImportAsync(
    $"COPY ... FROM STDIN (FORMAT BINARY)");

while (await reader.ReadAsync())      // 逐列讀取來源
```

```
{
    await importer.StartRowAsync();
    for (var i = 0; i < reader.FieldCount; i++)
        await importer.WriteAsync(reader.GetValue(i));  // 逐欄寫入目標
}
await importer.CompleteAsync();
```

這是真正的**串流模式**——來源的每一列讀取後立即寫入目標，記憶體中只存在一列的資料。即使是數百萬列的大型資料表，記憶體使用量也保持恆定。

14.3.3 記憶體使用比較

方式	記憶體使用	適用場景
`SELECT * → List → INSERT`	O(n) — 全部載入	小型資料表
`SqlBulkCopy + DataReader`	O(batch) — 批次大小	SQL Server
`COPY + 串流`	O(1) — 一列	PostgreSQL

14.4 優雅的部分失敗處理

DbCopy 的設計原則是：**非關鍵操作的失敗不應該中斷整體流程**。

14.4.1 索引建立的容錯

```
var indexSqls = await GetTableIndexDefinitionsAsync(sourceConnectionString, obj);
foreach (var idx in indexSqls)
{
    try
    {
        await conn.ExecuteWithLogAsync(logger, idx);
    }
    catch (Exception ex)
    {
        // 記錄警告，但繼續處理下一個索引
        logger.LogWarning(ex, "Failed to create index for {Schema}.{Name}",
            obj.Schema, obj.Name);
    }
}
```

為什麼索引失敗可以忽略？索引只影響查詢效能，不影響資料正確性。一個索引失敗不應該導致其他索引也無法建立。

14.4.2 外鍵建立的特殊處理

```
// SQL Server
catch (SqlException ex) when (ex.Number == 4902)
{
    // Error 4902: 參考的物件不存在
    logger.LogInformation("Skipped foreign key - referenced object does not exist");
}

// PostgreSQL
catch (PostgresException ex) when (ex.SqlState == "42P01")
{
    // 42P01: undefined_table
    logger.LogInformation("Skipped foreign key - referenced table does not exist");
}
```

為什麼用 `LogInformation` 而不是 `LogWarning`？因為這是**預期行為**——當使用者只複製部分資料表時，外鍵參考的資料表可能不在目標中。這不是錯誤，而是正常的操作結果。

14.4.3 行數查詢的容錯

```
async Task<long?> SafeGetRowCount(IDbService svc, string cs, DbObject o, string side)
{
    try { return await svc.GetRowCountAsync(cs, o); }
    catch (Exception ex)
    {
        logger.LogWarning(ex, "Failed to get {Side} row count ...", side, ...);
        return null;  // 回傳 null 而不是拋出例外
    }
}
```

回傳 `null` 的語意：前端 UI 會將 null 顯示為 ?，讓使用者知道行數未知，但不影響其他功能。

14.4.4 IDENTITY/序列重設的容錯

```
// SQL Server
try { await reseedCmd.ExecuteNonQueryAsync(); }
catch (Exception ex)
{
    logger.LogWarning(ex, "Failed to reseed identity for {Schema}.{Name}", ...);
```

```
}

// PostgreSQL
try { await targetConn.ExecuteWithLogAsync(logger, setValSql); }
catch (Exception ex)
{
    logger.LogWarning(ex, "Failed to reseed sequence for {Schema}.{Name}", ...);
}
```

序列重設失敗不影響已複製的資料，只會影響**後續的 INSERT 操作**。記錄警告讓使用者知道需要手動處理。

14.4.5 容錯層級總結

操作	失敗處理	日誌等級	繼續執行？
結構建立（Phase 1）	拋出例外	Error	否
資料複製（Phase 2）	拋出例外	Error	否
索引建立（Phase 3）	捕捉，繼續	Warning	是
外鍵建立（Phase 4）	捕捉，繼續	Warning/Info	是
行數查詢	捕捉，回傳 null	Warning	是
索引查詢	捕捉，回傳空列表	Warning	是
IDENTITY/序列重設	捕捉，繼續	Warning	是

下一章預告：最後一章將從整體視角回顧四階段複製流程的設計哲學——為什麼要分階段、每個階段解決什麼問題、以及相依性排序的策略。

15 四階段複製流程的設計哲學

這是本書的最後一章，也是最重要的一章。我們將從高層次回顧 DbCopy 最核心的設計決策——**四階段複製流程**——以及背後的考量。

15.1 為什麼分階段

如果只是單純地「把來源的東西複製到目標」，為什麼不一步到位，而要分成四個階段？

15.1.1 一步到位的問題

假設我們用以下方式複製資料表 `Orders`：

```
-- 一步到位
CREATE TABLE Orders (..., CONSTRAINT FK_Orders_Users FOREIGN KEY (UserId) REFERENCES Users(Id))
INSERT INTO Orders SELECT * FROM source.Orders;
CREATE INDEX IX_Orders_Date ON Orders(OrderDate);
```

這看起來很簡單，但在實際場景中會遇到大量問題：

問題 1：外鍵的雞生蛋問題

```
Orders.UserId → Users.Id  (Orders 依賴 Users)
Users.CreatedBy → Users.Id  (Users 自參照)
```

如果 `CREATE TABLE Orders` 包含外鍵定義，而 `Users` 表還沒建立，就會失敗。

問題 2：大量資料的索引效能

先建立索引再插入 100 萬筆資料，每一筆 INSERT 都需要更新索引。如果先插入資料再建立索引，只需要一次性掃描並建立索引，效能可以提升 10-100 倍。

問題 3：部分重試的困難

如果索引建立失敗了，是否需要重新複製整個資料表（包括 100 萬筆資料）？當然不需要。但如果所有操作綁在一起，就沒辦法只重試失敗的部分。

15.1.2 分階段的優勢

優勢	說明
解耦外鍵依賴	先建所有表的結構，再建外鍵，避免循環依賴
效能最佳化	先插入資料再建索引，大幅提升批次載入效率
部分重試	結構建好了？跳過。資料複製完了？只做索引。
進度可見	使用者可以看到目前在哪個階段
靈活控制	使用者可以選擇只執行特定階段

15.2 Phase 1：結構

目的：在目標資料庫建立物件的結構定義。

包含：- 資料表的欄位定義、主鍵 - 檢視的 SELECT 定義 - 預存程序的原始碼 - 函數的原始碼 - 使用者定義型別的定義 - 序列的定義

不包含：- 非主鍵索引（Phase 3）- 外鍵條件約束（Phase 4）- 資料列（Phase 2）

15.2.1 為什麼主鍵在 Phase 1 而不是 Phase 3？

主鍵是資料表定義的一部分——它定義了唯一性約束，影響資料的正確性。把主鍵放在 `CREATE TABLE` 語句中是標準做法：

```
CREATE TABLE Users (
    Id INT IDENTITY(1,1) NOT NULL,
    Name NVARCHAR(100) NOT NULL,
    CONSTRAINT PK_Users PRIMARY KEY (Id)    -- 主鍵在此
)
```

非主鍵索引是效能輔助，可以延後建立。

15.2.2 Phase 1 的前置條件

在執行 Phase 1 之前，`DbController` 會先做兩件事：

```
// 1. 確保 Schema 存在
await targetService.EnsureSchemaExistsAsync(request.Target.ConnectionString,
    request.Object.Schema);

// 2. 確認物件不存在
var exists = await targetService.CheckObjectExistsAsync(request.Target.ConnectionString,
```

```
    request.Object);
if (exists) return BadRequest("Object already exists in destination.");
```

15.3 Phase 2：資料

目的：將來源資料表的所有資料列複製到目標資料表。

使用技術：- SQL Server：`SqlBulkCopy` - PostgreSQL：`COPY ... FROM STDIN`

15.3.1 為什麼 Phase 2 在 Phase 3 之前？

```
有索引的情況下插入 100 萬筆：
每一筆 INSERT → 更新 B-Tree 索引 → O(N × log N)

無索引的情況下插入 100 萬筆後建索引：
批量 INSERT → O(N)
一次性建索引 → O(N log N)
總計 → O(N + N log N)  O(N log N)
```

但常數係數差異巨大：逐筆更新索引的 I/O 遠大於一次性排序建索引。

實務上，先資料再索引的方式通常快 5-50 倍。

15.3.2 Phase 2 的後處理

資料複製完成後，需要重設 IDENTITY/序列的種子值：

```
SQL Server: DBCC CHECKIDENT('table', RESEED)
PostgreSQL: setval('sequence', max_value)
```

否則後續的 INSERT 操作會從錯誤的起始值開始，導致主鍵衝突。

15.4 Phase 3：索引

目的：建立非主鍵的索引。

包含：- 唯一索引（UNIQUE INDEX）- 一般索引 - 含 INCLUDE 欄位的索引 - 複合索引

不包含：- 主鍵索引（已在 Phase 1 建立）

15.4.1 容錯設計

索引建立採用「逐一嘗試、失敗警告」的策略：

```
foreach (var idx in indexSqls)
{
    try { await conn.ExecuteWithLogAsync(logger, idx); }
    catch (Exception ex)
    {
        logger.LogWarning(ex, "Failed to create index ...");
        // 不中斷，繼續下一個索引
    }
}
```

索引失敗的常見原因：- 目標資料庫已有同名索引 - 索引定義包含了目標資料庫不支援的功能 - 資料不符合唯一索引的約束

15.5 Phase 4：外鍵

目的：建立外部索引鍵條件約束。

15.5.1 為什麼外鍵是最後一個階段？

外鍵是最「危險」的操作，因為它有外部依賴——參考的資料表必須已經存在且有正確的資料。

```
Orders.UserId → Users.Id

要建立這個外鍵，需要：
1. Orders 表已存在（Phase 1）
2. Users 表已存在（Phase 1）        ← 可能不在這次複製範圍內！
3. Orders.UserId 的每個值在 Users.Id 中都存在（Phase 2）  ← 如果 Users 沒有資料？
```

把外鍵放在最後，確保了：- 所有資料表的結構都已建立 - 所有資料都已複製 - 所有索引都已建立（外鍵參考欄位通常有索引）

15.5.2 特殊錯誤處理

```
// SQL Server: Error 4902 = 參考物件不存在
catch (SqlException ex) when (ex.Number == 4902)
{
    logger.LogInformation("Skipped - referenced object does not exist (Error 4902)");
}

// PostgreSQL: 42P01 = undefined_table
catch (PostgresException ex) when (ex.SqlState == "42P01")
{
    logger.LogInformation("Skipped - referenced table does not exist (42P01)");
}
```

使用 `LogInformation`（而非 `LogWarning`）是因為這是**預期行為**：使用者可能只複製了部分資料表。

15.6 相依性排序與複製順序

15.6.1 列舉值蘊含的順序

回顧第 3 章的 `DbObjectType` 列舉：

```
public enum DbObjectType
{
    UserDefinedType,        // 0
    UserDefinedTableType,   // 1
    Sequence,               // 2
    Table,                  // 3
    View,                   // 4
    Procedure,              // 5
    Function                // 6
}
```

前端 UI 依據 Schema → Type（列舉值）→ Name 排序，自然形成正確的複製順序。

15.6.2 建議的複製順序

第一輪：基礎型別

1. `UserDefinedType` (Enum, Domain, Composite)
2. `UserDefinedTableType`

第二輪：獨立物件

3. `Sequence`

```
第三輪：資料容器
  4. Table (Phase 1: 結構)
  5. Table (Phase 2: 資料)
  6. Table (Phase 3: 索引)

第四輪：程式碼物件
  7. View
  8. Procedure
  9. Function

第五輪：關聯
  10. Table (Phase 4: 外鍵)
```

15.6.3 相依性資訊的呈現

Compare 端點會回傳每個物件的依賴清單：

```
{
    "sourceObject": { "schema": "dbo", "name": "Orders", "type": 3 },
    "dependencies": ["dbo.Users", "dbo.PhoneNumber"],
    "existsInDestination": false,
    "status": "Pending"
}
```

前端可以利用這個資訊提醒使用者：「Orders 依賴 Users 和 PhoneNumber，請確保它們先被複製。」

15.6.4 完整流程圖

```
使用者選擇要複製的物件
          ↓
依照列舉值排序 (型別 → 序列 → 表 → 檢視 → 程序 → 函數)
          ↓

  對每個物件：

   EnsureSchemaExists (目標)
          ↓
   CheckObjectExists (目標)
          ↓
   如果是 Table：
     Phase 1: GetDefinition → Execute (CREATE)
     Phase 2: CopyTableData (BulkCopy/COPY)
     Phase 3: GetIndexDefs → Execute (各索引)
```

```
    Phase 4: GetFKDefs → Execute（各外鍵）

  如果是其他物件：
    Phase 1: GetDefinition → Execute

  更新 SyncStatus（Success / Error）

          ↓
顯示同步結果摘要
```

15.6.5 設計的自洽性

整個四階段複製流程的設計是**自洽的**：

1. **列舉排序**確保型別在資料表之前建立
2. **Phase 1** 確保所有結構在資料之前建立
3. **Phase 2** 確保所有資料在索引之前插入
4. **Phase 3** 確保所有索引在外鍵之前建立
5. **Phase 4** 在最後建立外鍵，此時所有前提都已滿足

任何一個環節的失敗都可以獨立重試，不影響已完成的環節。

15.7 本書總結

DbCopy 是一個小而精的工具——整個系統只有 7 個 C# 檔案、1 個 HTML 頁面、1 個 CSS 檔案。但在這精簡的體積中，蘊含了大量的設計決策：

- **介面導向設計**讓我們能夠支援多個資料庫引擎
- **Dapper 包裝層**確保了日誌的完整性
- **四階段複製**解決了外鍵依賴和效能問題
- **嵌入式資源**實現了零安裝部署
- **連接埠衝突偵測**提升了使用者體驗
- **結構化日誌**方便了偵錯和維運
- **平行化查詢**提升了比較操作的效能
- **優雅的部分失敗**確保了系統的健壯性

希望本書能幫助讀者理解——一個好的工具不需要複雜的架構，但需要每一個設計決策都經過深思熟慮。

> **最佳的程式碼不是最聰明的程式碼，而是最容易理解、最難出錯的程式碼。**